

Computer Organization & Architecture (BE04000251)



Unit-3 Basic Computer Organization and Design

**Vrunda Dave
Assistant Professor
IT Department
V.V.P. Engineering College, Rajkot**

Outline

- Instruction codes
- Computer registers
- Computer instructions
- Timing and Control
- Instruction cycle
- Register-Reference Instructions
- Memory-Reference Instructions
- Input-output and interrupt
- Interrupt Cycle
- Design of Basic computer
- Design of Accumulator Unit.

Instruction codes

Instruction codes

- **Program:**

- A program is a set of instructions that specify the operations, operands and the sequence by which instructions are to be executed.

- **Computer Instruction:**

- A computer instruction is a binary code that specifies a sequence of micro-operations for the computer.

- **Instruction Code:**

- An instruction code is a group of bits that instruct the computer to perform a specific operation.
- **Example:** ADD B

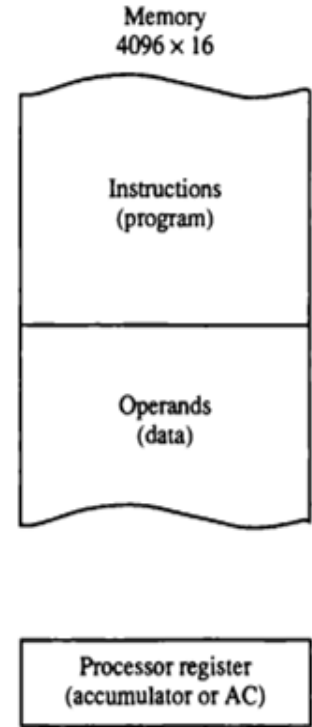
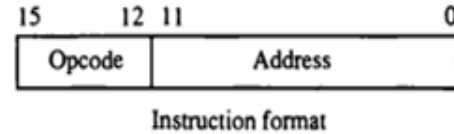
Instruction codes

- **Operation Code (Opcode):**

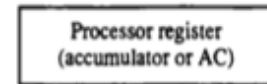
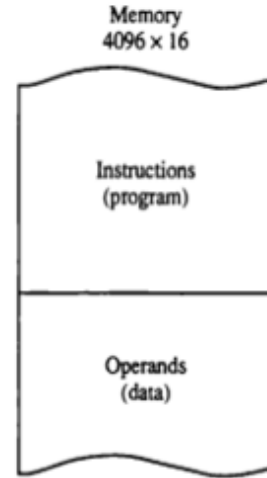
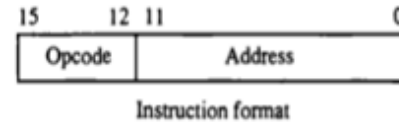
- The operation code of an instruction is a group of bits that define such operations as add, subtract, multiply, shift, and complement.
- **The number of bits required for the operation code of an instruction depends on the total number of operations available in the computer.**
- The operation code must consist of at least n bits for a given 2^n (or less) distinct operations.
- **Example:** consider a computer with 64 distinct operations, one of them being ADD operation. The operation code consists of six bits, with a bit configuration 110010 assigned to ADD operation.
- When this operation code is decoded in the control unit, the computer issues control signals to read an operand from memory and add the operand to a register.

Stored Program Organization

- The simplest way to organize a computer is to have **one processor register** and an **instruction code format with two parts**.
- The first part specifies the operation (**opcode**) to be performed and the second specifies an address (**operand**).
- The memory address tells the control where to find an operand in memory.
- This operand is read from memory and used as the data to be operated on together with the data stored in the processor register.



- Instructions are stored in one section of memory and data in another.
- For a memory unit with 4096 words, we need 12 bits to specify an address since $2^{12} = 4096$.
- If we store each instruction code in one 16-bit memory word, we have available four bits($2^4 = 16$) for operation code (opcode) to specify one out of 16 possible operations, and 12 bits to specify the address of an operand.
- The control reads a 16-bit instruction from the program portion of memory.
- It uses the 12-bit address part of the instruction to read a 16-bit operand from the data portion of memory.
- It then executes the operation specified by the operation code.

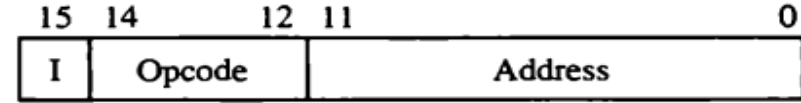


Example(Opcode):

0000 – ADD
 0001 – SUB
 0010 - CMA

Instruction format of basic computer

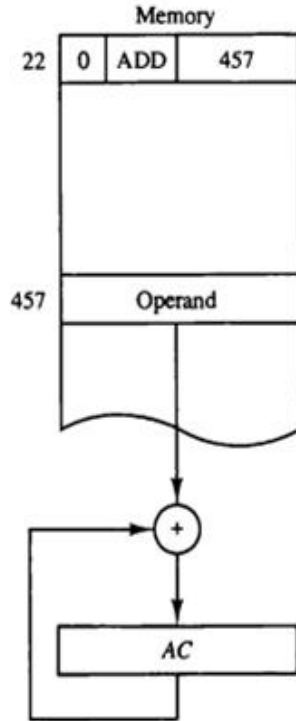
- It is a set of bits that specifies the operation to be performed.
- It Consist of a 3-bit operation code, a 12-bit address, and an address mode is designated by I.



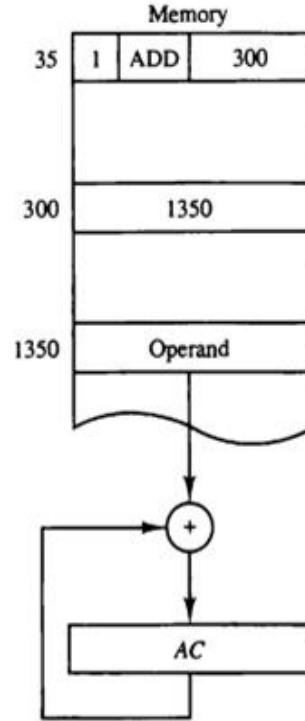
(a) Instruction format

- **Immediate operand:**
 - If the second part of an instruction format specifies an operand, the instruction is said to have a immediate operand. MVI R1, 11
- **Direct Address:**
 - If the second part of an instruction format specifies the address of an operand, the instruction is said to have a Direct address.
- **Indirect Address:**
 - In Indirect address, the bits in the second part of the instruction designate an address of a memory word in which the address of the operand is found.

Direct and Indirect Addressing of Memory



(b) Direct address



(c) Indirect address

- We define the **effective address** to be the address of the operand in a computation - type instruction or the target address in a branch type instruction.
- Effective address in the instruction of fig(b) is 457 and in the instruction of fig(c) is 1350.



- A **direct address** instruction is placed at address 22 in memory.
- The I bit is 0, so the instruction is recognized as a direct address instruction.
- The opcode specifies an ADD instruction, and the address part is the binary equivalent of 457.
- The control finds the operand in memory at address 457 and adds it to the content of AC.



- A **indirect address** instruction is placed at address 35 in memory.
- The I bit is 1, so the instruction is recognized as an indirect address instruction.
- The address part is the binary equivalent of 300.
- The control goes to address 300 to find the address of the operand.
- The address of the operand in this case is 1350.
- The operand found in address 1350 is then added to the content of AC.

Computer Registers

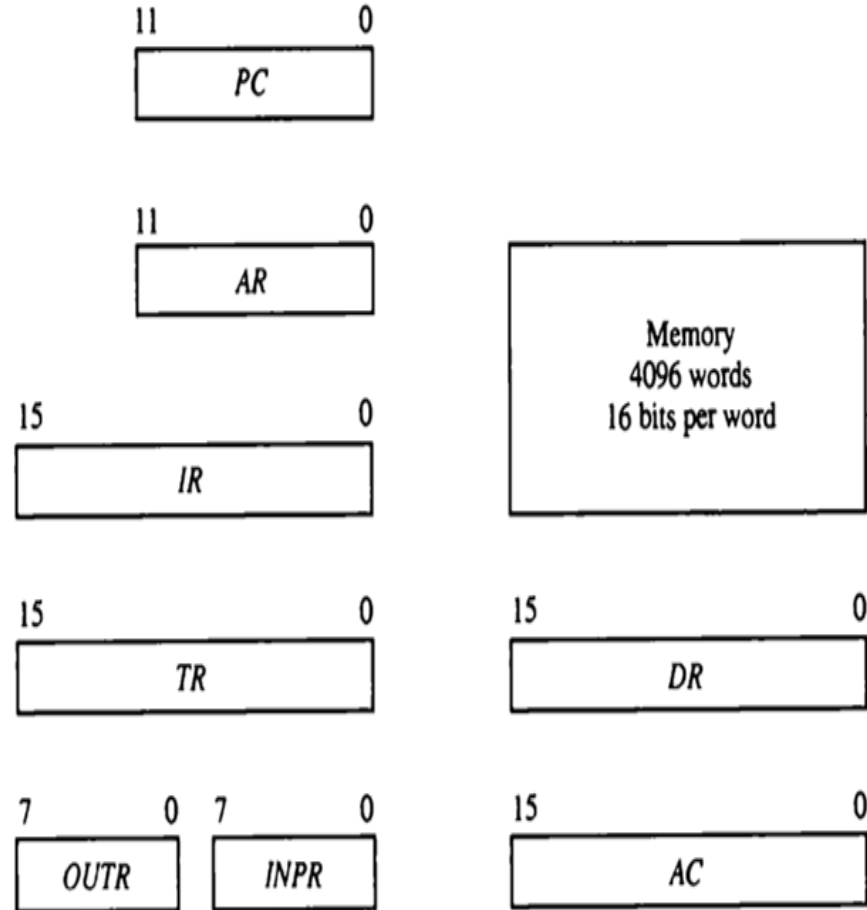
Computer Registers

- Computer instructions are normally stored in consecutive memory locations and are executed sequentially one at a time. The control reads an instruction from a specific address in memory and executes it. It then continues by reading the next instruction in sequence and executes it and so on.
- This type of instruction needs a counter that calculate the address of next instruction(PC).
- It is also necessary to provide a register in the control unit for storing the instruction code after it is read from memory.
- The computer needs processor registers for manipulating data and a register for holding a memory address.

Register symbol	Number of bits	Register name	Function
<i>DR</i>	16	Data register	Holds memory operand
<i>AR</i>	12	Address register	Holds address for memory
<i>AC</i>	16	Accumulator	Processor register
<i>IR</i>	16	Instruction register	Holds instruction code
<i>PC</i>	12	Program counter	Holds address of instruction
<i>TR</i>	16	Temporary register	Holds temporary data
<i>INPR</i>	8	Input register	Holds input character
<i>OUTR</i>	8	Output register	Holds output character

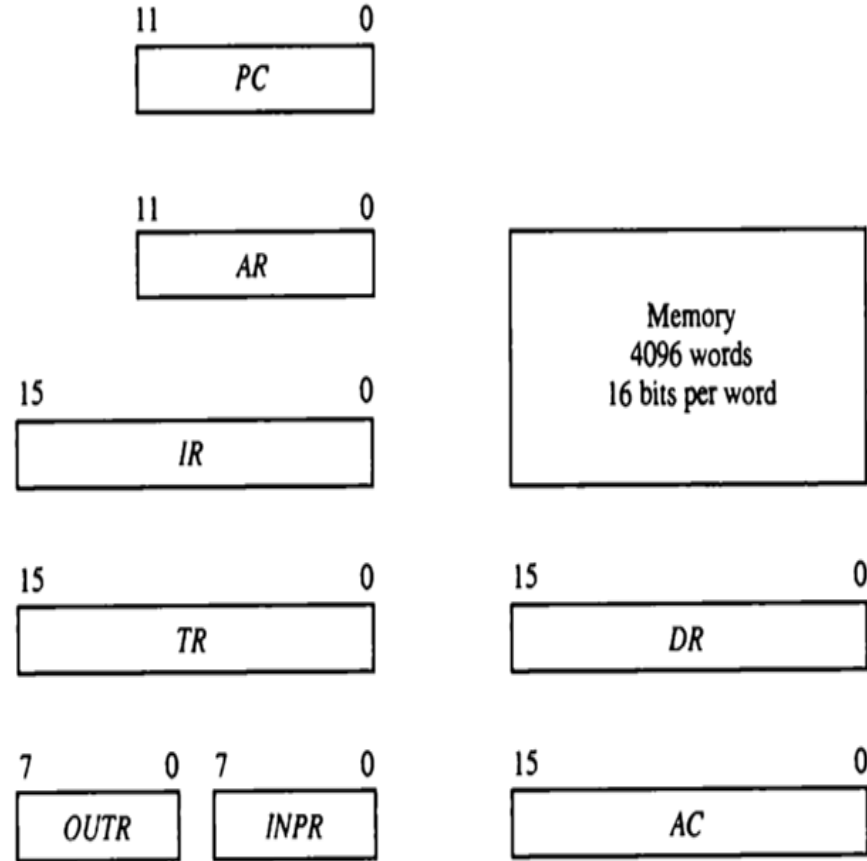
Computer Registers

- **Memory unit:** Capacity of 4096 words & each word contains 16 bits. 12 bits, specify the address of an operand.
- **Data register (DR):** holds the operand read from memory.
- **Accumulator (AC):** It is a general-purpose processing register.
- **Instruction Register (IR):** Instruction read from memory is placed in IR
- **Temporary register (TR):** Used for holding temporary data during the processing.



Computer Registers

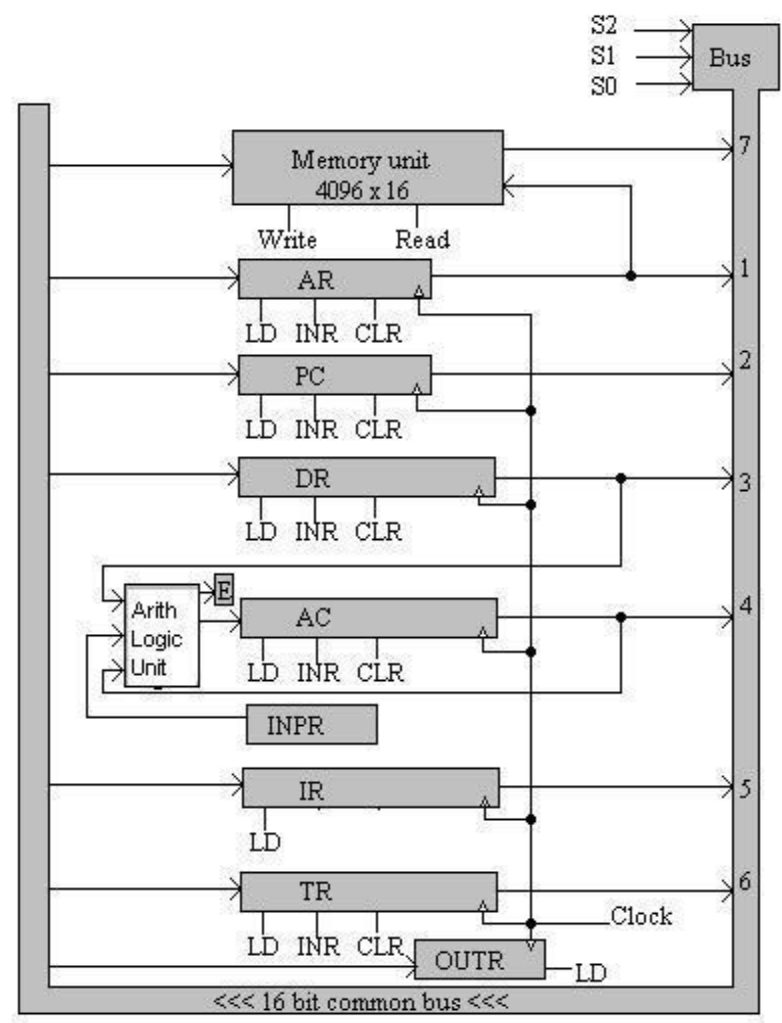
- **Address Register (AR):** It has 12 bits because the width of a memory address is 12-bits.
- **Program counter (PC):** It is of 12 bits & holds the address of the next instruction to be read (fetched) from memory after the current instruction is executed. PC is incremented by one, so it holds the address of the next instruction in sequence.
- **INPR & OUTR:** Two registers are used for input and output. The input register (INPR) receives an data from an input device. The output register (OUTR) holds data for output devices.



Common Bus system

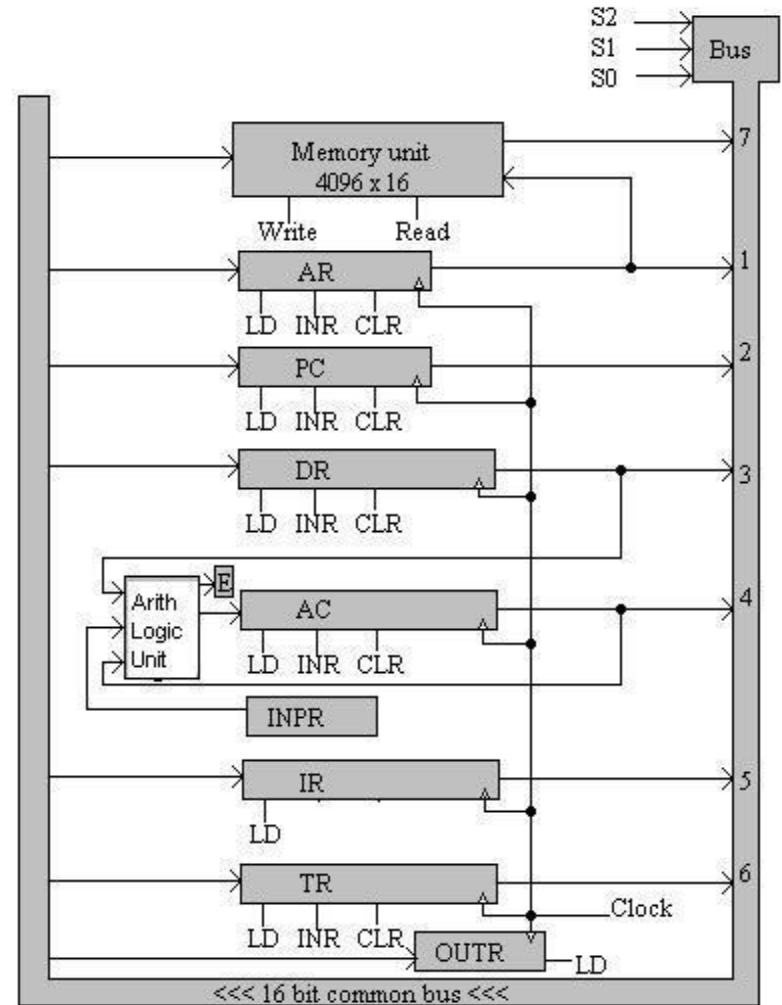
Common Bus System

- Paths must be provided **to transfer information from one register to another and between memory and registers**. Efficient scheme for transferring information in a system with many registers is to use **a common bus**.
- The outputs of registers and memory are connected to the common bus. The specific output is selected for the bus lines at any given time are determined from the binary value of the selection variables **S2** , **S1** , and **S0** .
- For Eg, the number along the output of DR is 3. The 16-bit outputs of DR are placed on the bus lines when $S_2S_1S_0 = 011$.



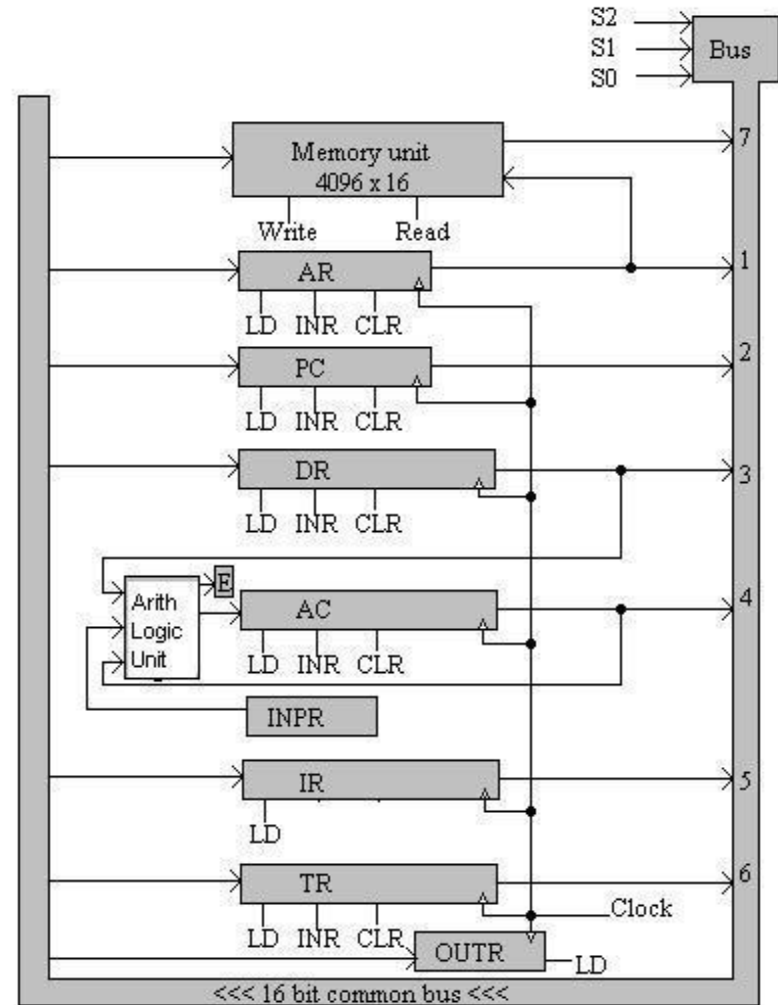
Common Bus System

- The lines from the common bus are connected to the inputs of each register and the data inputs of the memory.
- The **particular register whose LD(load) input is enabled receives the data from the bus.**
- The memory receives the contents of the bus when its write input is activated. The memory places its 16-bit output onto the bus when the read input is activated and $S_2S_1S_0 = 111$.
- Four registers DR, AC, IR, and TR, have 16 bits each. Two registers AR and PC, have 12 bits each because they hold a memory address.



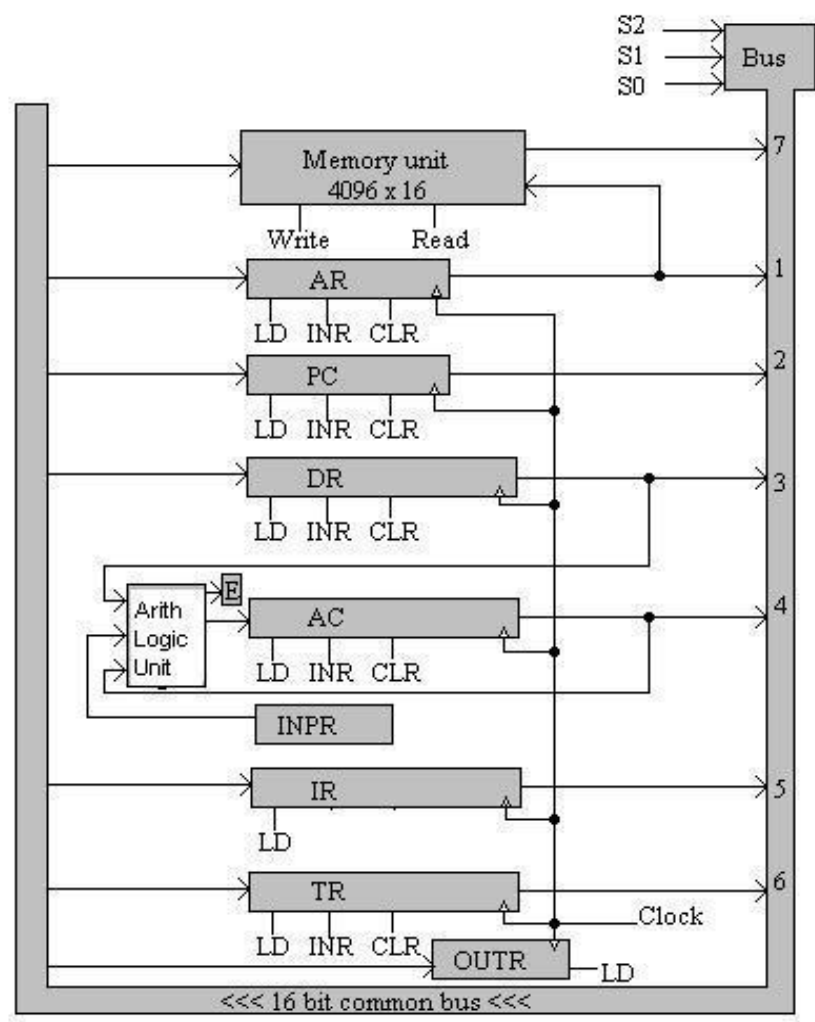
Common Bus System

- When the contents of AR or PC are applied to the 16-bit common bus, the four most significant bits are set to 0's.
- When AR or PC receives information from the bus, only the 12 least significant bits are transferred into the register.
- The input register INPR and the output register OUTR have 8 bits each and communicate with the eight least significant bits in the bus. INPR is connected to provide information to the bus but OUTR can only receive information from the bus.



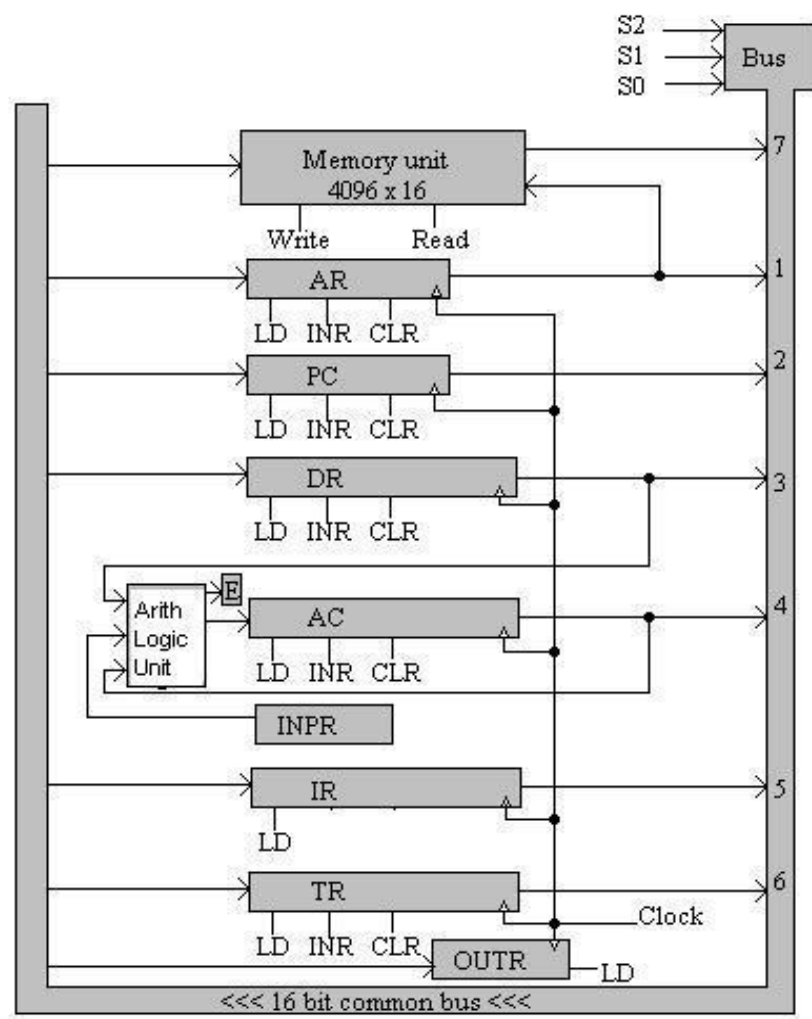
Common Bus System

- This is because INPR receives a character from an input device which is then transferred to AC . OUTR receives a character from AC and delivers it to an output device.
- The 16 lines of the common bus receive information from six registers and the memory unit.
- Five registers have three control inputs: LD (load), INR (increment), and CLR (clear).



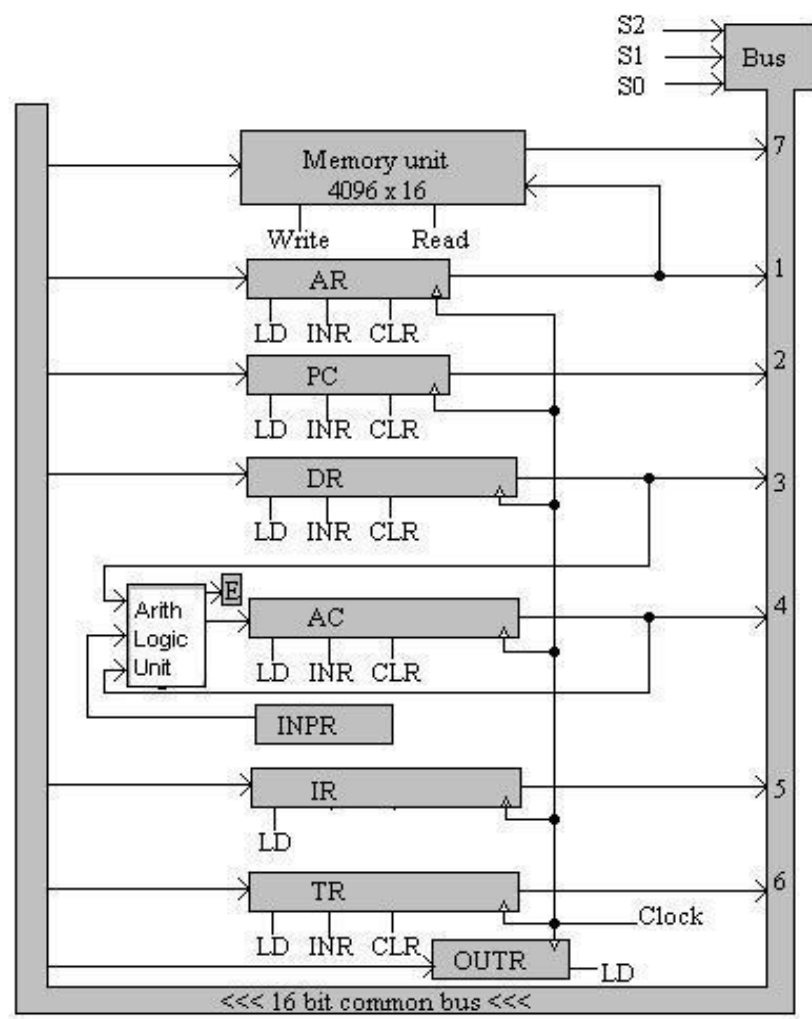
Common Bus System

- The input data and output data of the memory are connected to the common bus, but the memory address is connected to AR. Therefore, AR must always be used to specify a memory address.
- The content of any register can be specified for the memory data input during a write operation. Similarly any register can receive the data from memory after a read operation except AC.
- The 16 inputs of AC come from an adder and logic circuit. This circuit has three sets of inputs.



Common Bus System

- One set of 16-bit inputs come from the outputs of AC. They are used to implement register micro-operations such as complement AC and shift AC.
- Another set of 16-bit inputs come from the data register DR.
- Example: The inputs from AC and DR are used for arithmetic and logic microoperations. The result of the operation is transferred to AC and the end carry is transferred to flip-flop E(Extended AC bit).
- A third set of 8-inputs come from the input register INPR.



Common Bus System

- Examples:

->PC to bus(010)

->bus to AR(LD input)

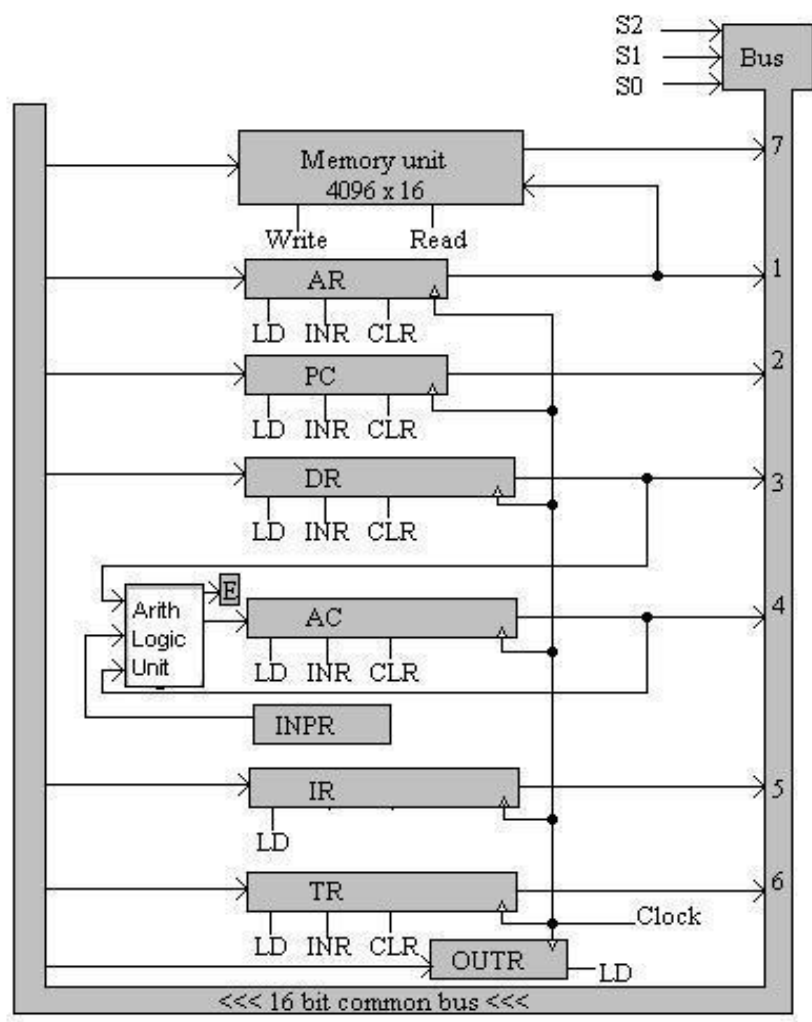
->AR to Memory Unit(to fetch instruction)

->Memory Unit to bus(111 and read signal)

->bus to DR(LD input)

->AC to bus(100)

->bus to OUTR(LD input)



Common Bus System

- Examples:

->PC to bus(010)

->bus to AR(LD input)

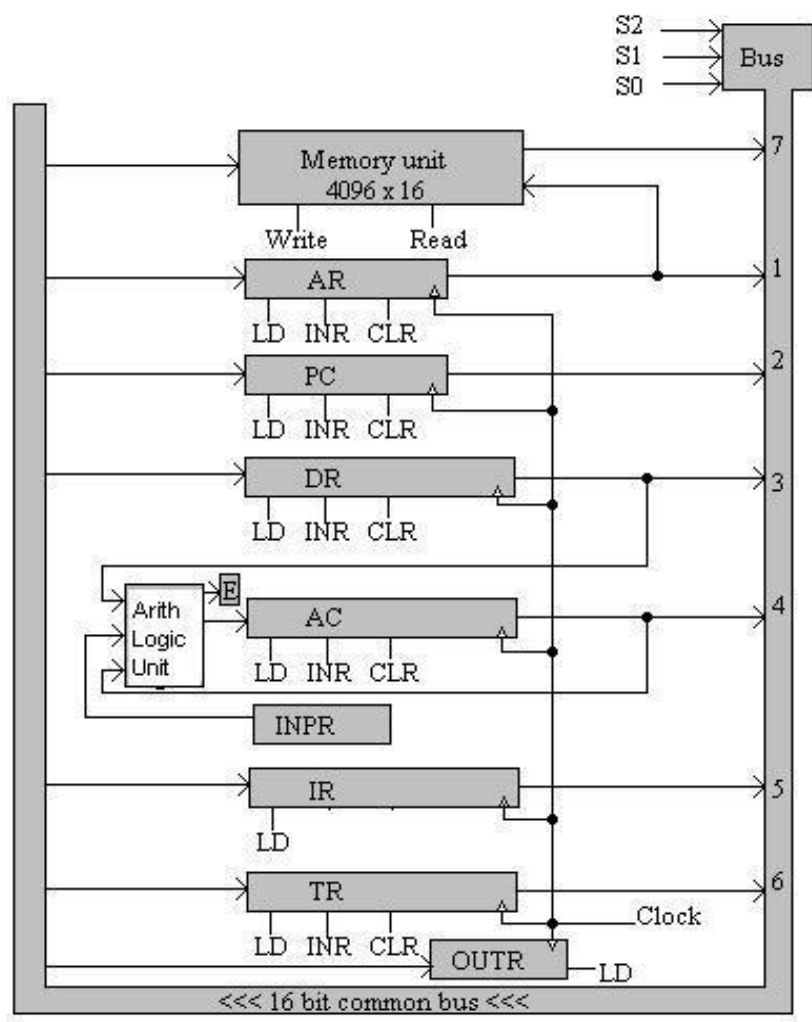
->AR to Memory Unit(to fetch instruction)

->Memory Unit to bus(111 and read signal)

->bus to DR(LD input)

->AC to bus(100)

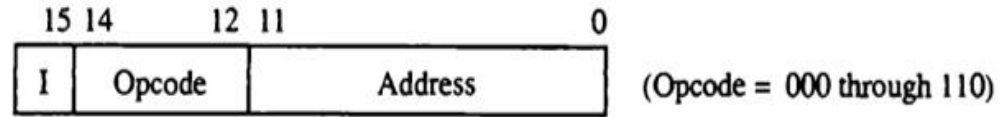
->bus to OUTR(LD input)



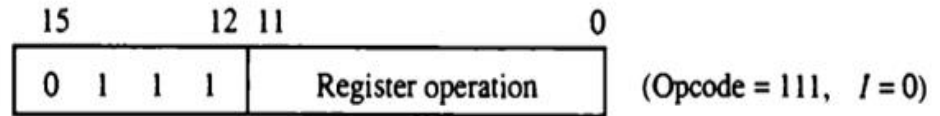
Computer Instruction

Computer Instruction Format

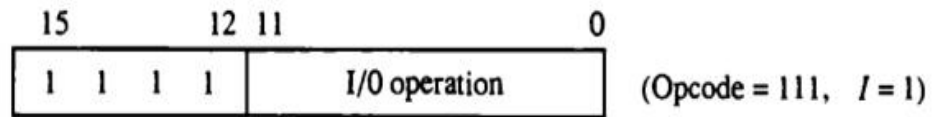
- The basic computer has three instruction code formats.
- The operation code(opcode) part of the instruction contains three bits and the remaining 13 bits used to decide the type of operation.



(a) Memory – reference instruction



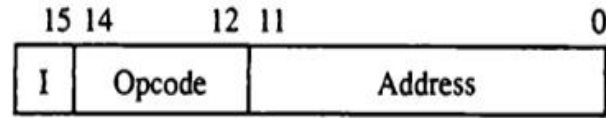
(b) Register – reference instruction



(c) Input – output instruction

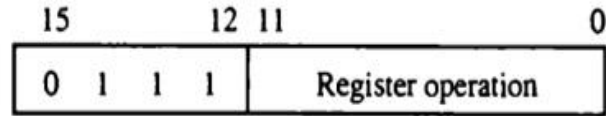
Computer Instruction Format

- **Memory reference instruction:** Uses 12 bits to specify an address and one bit to specify the addressing mode I. I = 0 for direct address and I=1 for indirect address.
- **Register reference instructions:** It is recognized by the operation code 111 with a 0 in the leftmost bit (bit 15) of the instruction. The other 12 bits are used to specify the operation.
- **Input output instruction:** It is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction.



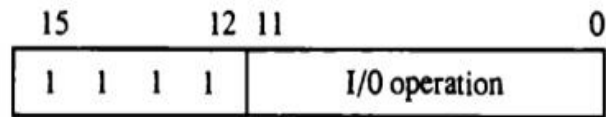
(Opcode = 000 through 110)

(a) Memory – reference instruction



(Opcode = 111, I = 0)

(b) Register – reference instruction

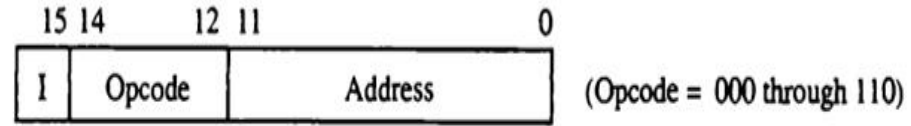


(Opcode = 111, I = 1)

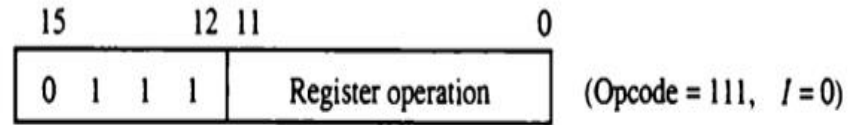
(c) Input – output instruction

Computer Instruction Format

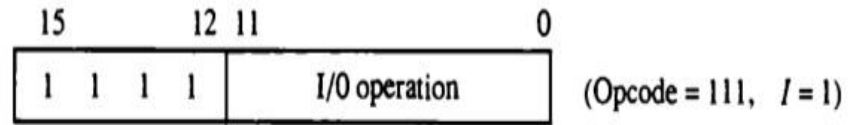
- If the three opcode bits in positions 12 through 14 are not equal to 111, the instruction is a memory-reference type and the bit in position 15 is taken as the addressing mode I.
- If the 3-bit opcode is equal to 111, control then check the bit in position 15. If this bit is 0, the instruction is a register-reference type. If this bit is 1, the instruction is an input-output type.



(a) Memory – reference instruction



(b) Register – reference instruction



(c) Input – output instruction

Note: I is not used as a mode bit when the operation code is equal to 111.

Basic Computer Instructions

TABLE 5-2 Basic Computer Instructions

Symbol	Hexadecimal code		Description
	<i>I</i> = 0	<i>I</i> = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC positive
SNA	7008		Skip next instruction if AC negative
SZA	7004		Skip next instruction if AC zero
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

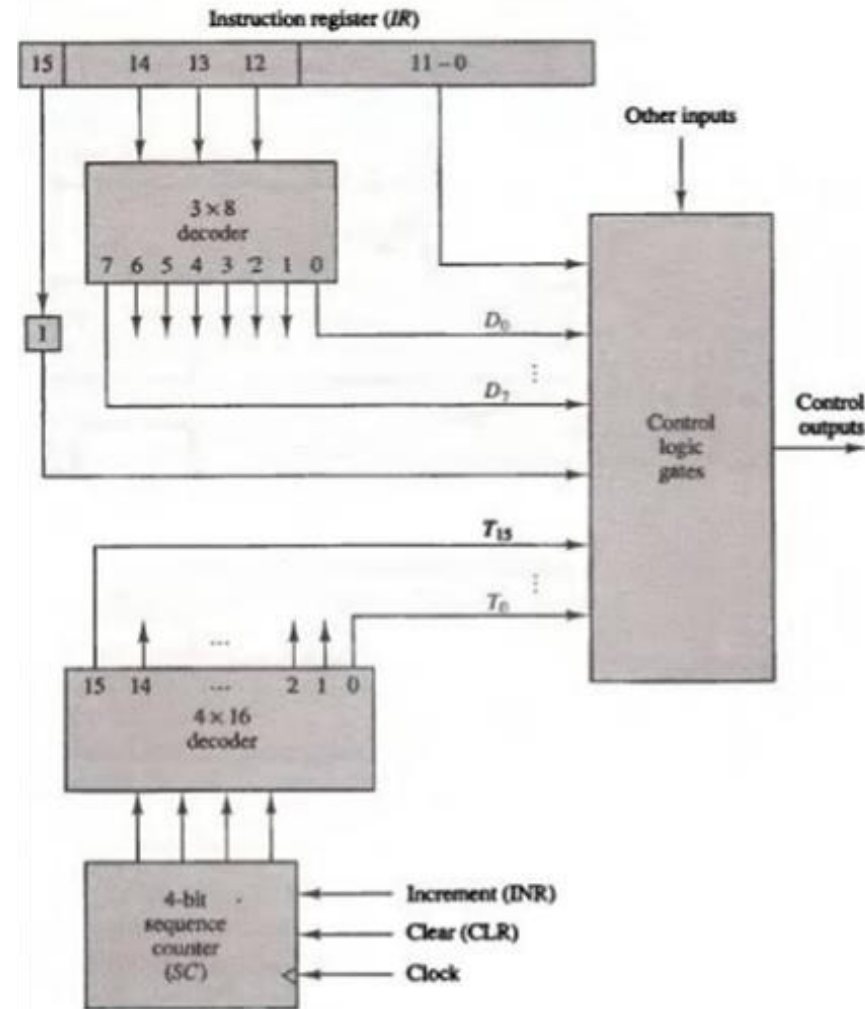
Timing and Control

Timing and Control Unit

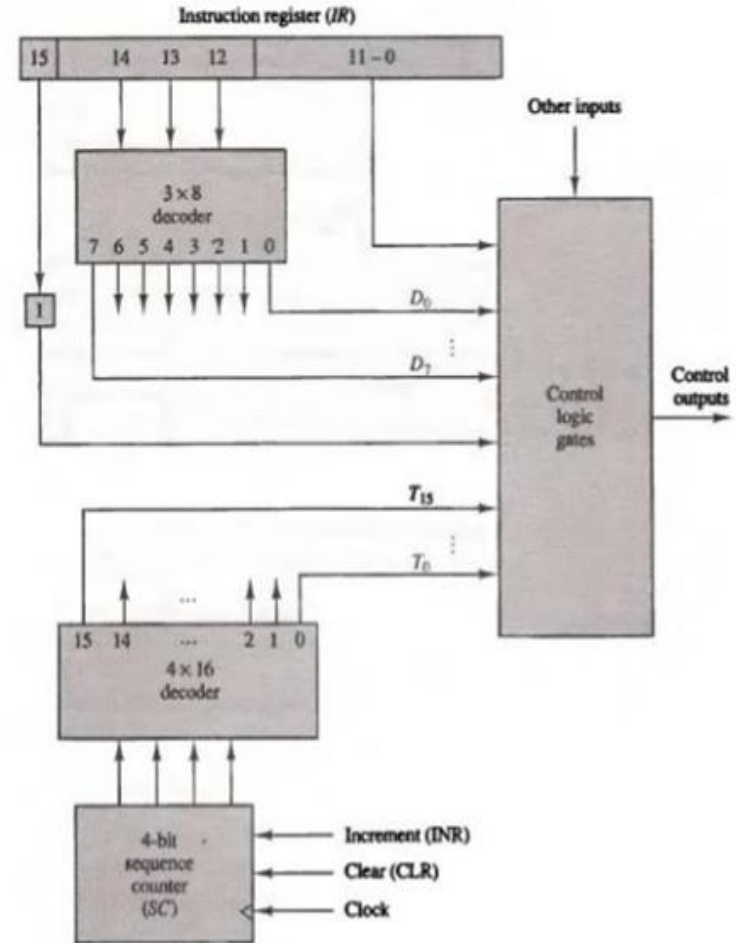
- Timing for all registers is controlled by a master clock.
 - Clock pulses are applied to all flip-flops and registers.
 - We have control unit present in a CPU in order to generate timing signals and control signals.
-
- We can design control unit in two ways:
- 1. Hardwired Control Unit:**
 - Control units will be designed using hardware components like logic gates, flip-flops, decoders, registers and other logic circuits.
 - Difficult to modify(as we have to change wiring).
 - Extremely Faster.
 - 2. Microprogrammed Control Unit(will see in upcoming chapter):**
 - Microprogrammed control unit generates the control signals with the help of micro instructions stored in control memory.
 - Modification is easy.

Timing and Control Unit (Hardwired)

- It consists of two decoders, a sequence counter and a number of control logic gates.
- **An Instruction read from memory is placed in the instruction register (IR).**
- The instruction register is divided into three parts: **the I bit, the operation code, and bits 0 through 11.**
- **The operation code in bits 12 through 14 (3 bits then $2^3 = 8$ total operations) are decoded with a 3 x 8 decoder.** The eight outputs of the decoder are D_0 through D_7 .
- **Example:** 001 means ADD operation and whenever 001 is as opcode the D_1 is active. So now control unit instruct the CPU to perform ADD operation.



- Address bits(0-11) are passed to control logic gates.
- Bit 15 of the instruction is transferred to a flip-flop designated by the symbol I.
- The 4-bit(total 16 combinations) sequence counter (SC) can count in binary from 0 through 15. **The outputs of the counter are decoded into 16 timing signals T_0 through T_{15} .**
- **Example:** When the AND operation is performed T_0 may be active. When the ADD operation is performed T_1 may be active.
- Sequence Counter has three inputs (INR, CLR, Clock).
- Whenever INC input is enabled as well as clock pulse is applied then only we will get next timing signal. ($T_0, T_1, T_2 \dots$).
- Whenever we will enable CLR input signal then we will get starting timing signal(T_0).



Timing and Control Unit

- **Timing Diagram ($D_3T_4 : SC \leftarrow 0$)**
- When D_3 signal is active and T_4 is active then Sequence Counter is cleared.
- The sequence counter SC responds to the positive transition of the clock. Initially, the CLR input of SC is active.
- The first positive transition of the clock clears SC to 0, which in turn activates the timing T_0 out of the decoder. T_0 is active during one clock cycle. The positive clock transition labeled T_0 in the diagram will trigger only those registers whose control inputs are connected to timing signal T_0 .
- SC is incremented with every positive clock transition, unless its CLR input is active.

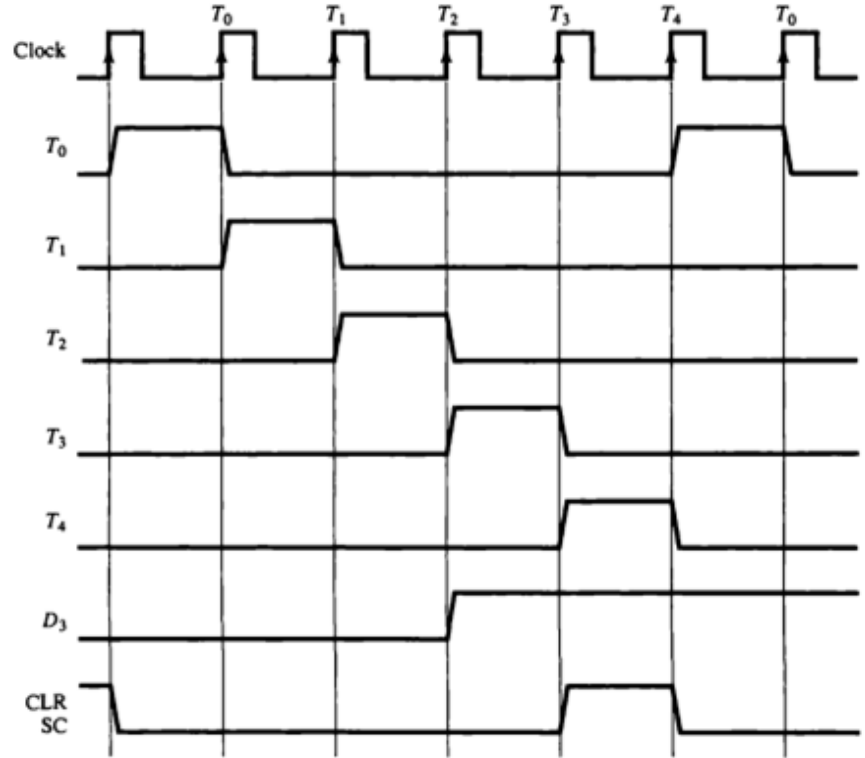


Figure 5-7 Example of control timing signals.

Timing and Control Unit

- This procedure generates the sequence of timing signals T_0 , T_1 , T_2 , T_3 and T_4 , and so on. If SC is not cleared, the timing signals will continue with T_5 , T_6 , up to T_{15} and back to T_0 .
- The last three waveforms show how SC is cleared when $D_3T_4 = 1$. Output D_3 from the operation decoder becomes active at the end of timing signal T_2 . When timing signal T_4 becomes active, the output of the AND gate that implements the control function D_3T_4 becomes active.
- This signal is applied to the CLR input of SC. On the next positive clock transition the counter is cleared to 0. This causes the timing signal T_0 to become active instead of T_5 that would have been active if SC were incremented instead of cleared.

Instruction cycle

Instruction Cycle

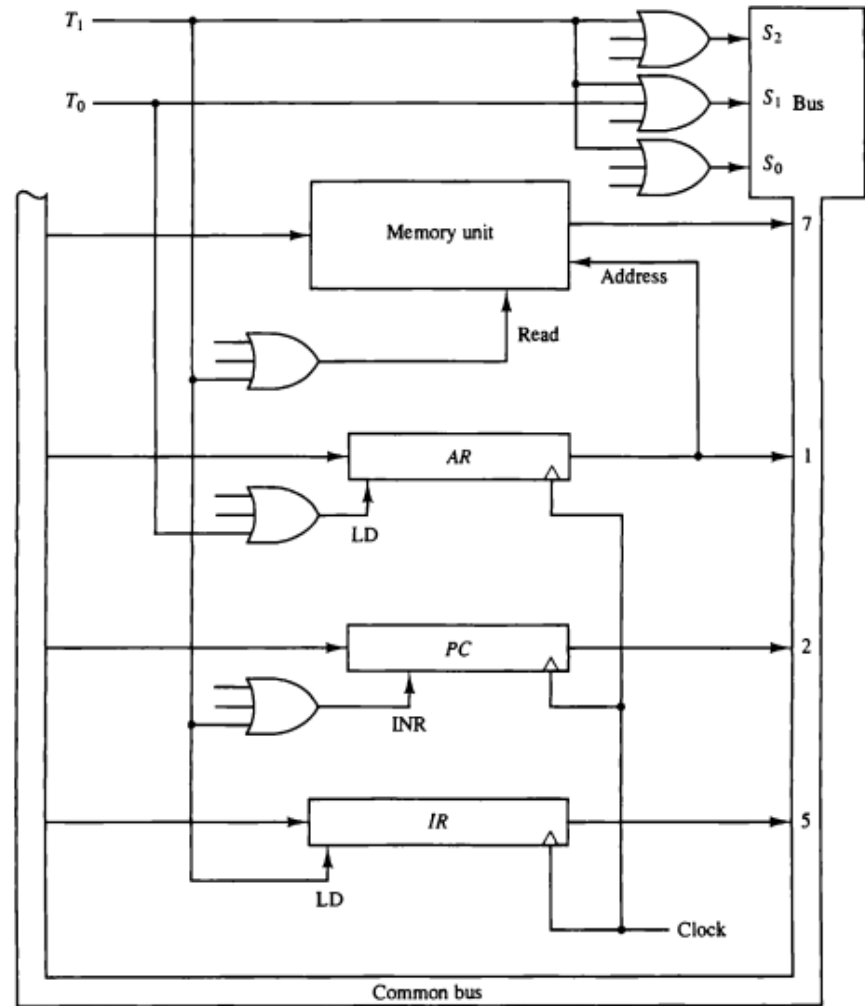
- A program residing in the memory unit of the computer consists of a sequence of instructions.
- The program is executed in the computer by going through a cycle for each instruction.
- Each instruction cycle consists of the following phases:
 1. Fetch an instruction from memory.
 2. Decode the instruction(Determining which operation is to be executed).
 3. Read the effective address from memory if the instruction has an indirect bit.
 4. Execute the instruction.
- Upon the completion of step 4, the control goes back to step 1 to fetch, decode, and execute the next instruction.
- This process executes continuously unless a HALT instruction is encountered.

Register transfer operation for Fetch and Decode

- Initially, the program counter PC is loaded with the address of the first instruction in the program.
- The sequence counter SC is cleared to 0, providing a decoded timing signal T_0 .
- After each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T_0, T_1, T_2 , and so on.
- Microoperations for the fetch and decode phases:**
 - Fetching** $T_0 : AR \leftarrow PC$
 - Fetching** $T_1 : IR \leftarrow M[AR], PC \leftarrow PC + 1$
 - Decoding** $T_2 : AR \leftarrow IR(0-11), D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14), I \leftarrow IR(15)$
- Since only AR is connected to the address inputs of memory with timing signal T_0 .
- The instruction read from memory is then placed in the instruction register IR with the clock transition associated with timing signal T_1 . At the same time, PC is incremented by one to prepare it for the address of the next instruction in the program.
- At time T_2 , the operation code in IR is decoded, the indirect bit is transferred to flip-flop I, and the address part of the instruction is transferred to AR. Note that SC is incremented after each clock pulse to produce the sequence T_0, T_1 and T_2 .

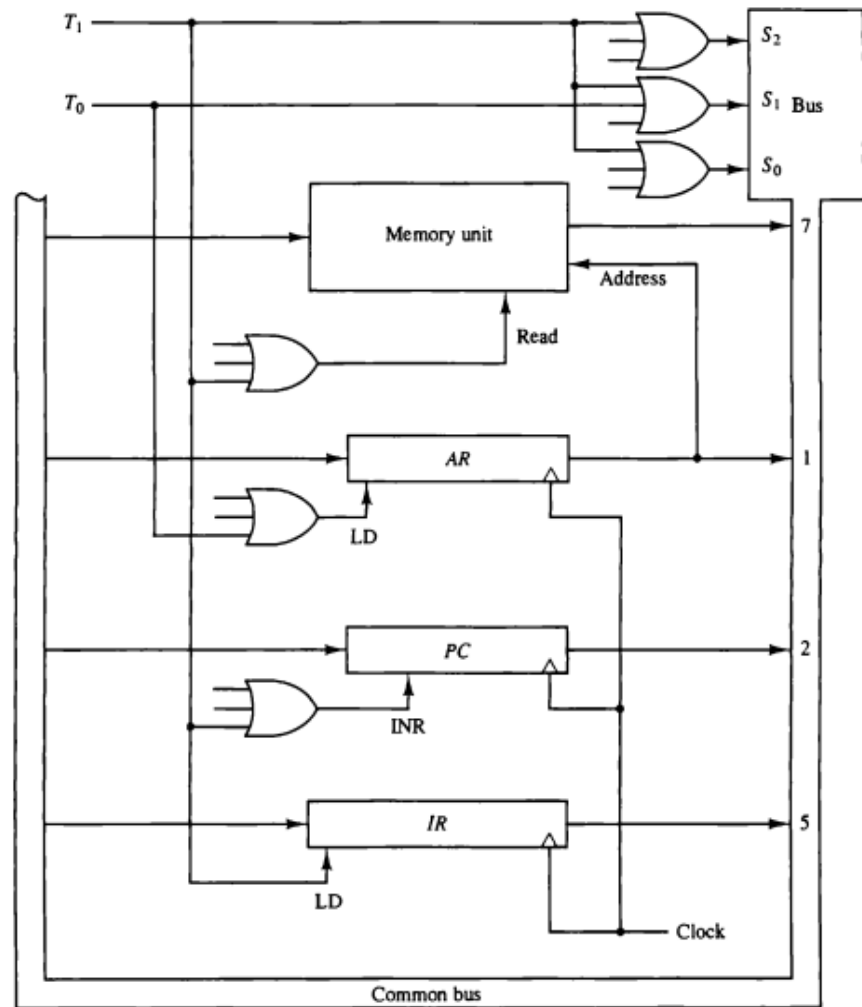
Register transfer for Fetch phase

- Figure shows how first two register transfer statements (T_0 and T_1) are implemented in bus system.
 - In order to implement the following statement T_0 : **AR $\leftarrow$ PC.**
 - It is necessary to use timing signal T_0 to provide the following connections in the bus system.
- Place the content of PC onto the bus by making the bus selection inputs $S_2S_1S_0$ equal to 010.
 - Transfer the content of the bus to AR by enabling the LD input of AR.
- The next clock transition initiates the transfer from PC to AR since $T_0=1$.



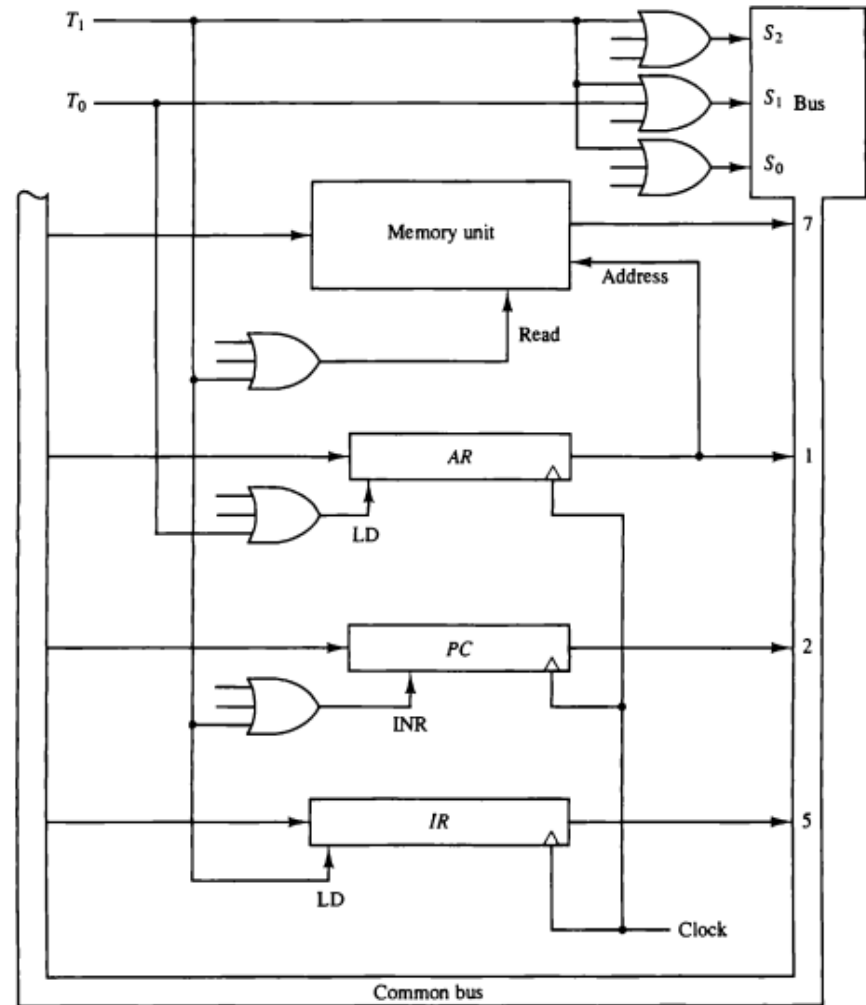
Register transfer for Fetch phase

- In order to implement the second statement $T_1 : IR \leftarrow -M[AR], PC \leftarrow -PC + 1$.
- It is necessary to use timing signal T_1 to provide the following connections in the bus system.
 1. Enable the read input of memory.
 2. Place the content of memory onto the bus by making $S_2S_1S_0 = 111$.
 3. Transfer the content of the bus to IR by enabling the LD input of IR
 4. Increment PC by enabling the INR input of PC.
- The next clock transition initiates the read and increment operations since $T_1 = 1$.



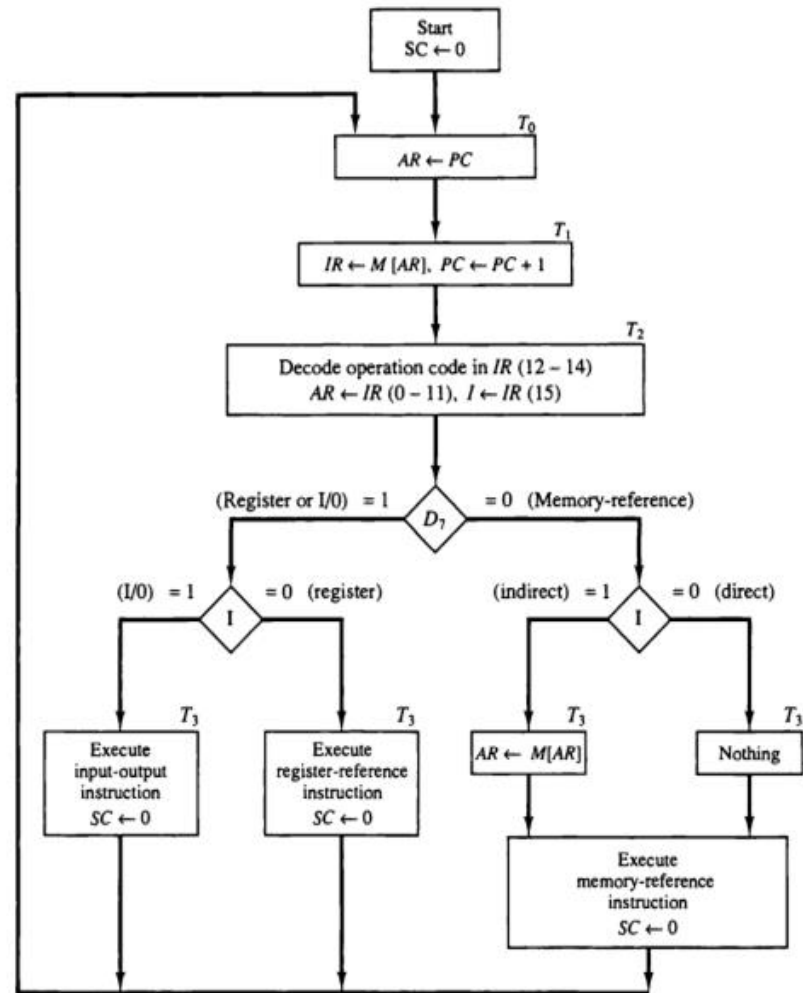
Register transfer for Fetch phase

- The timing signal that is active after the decoding is T_3 . **During time T_3** , the control unit **determines the type of instruction(instruction cycle)** that was just read from memory.
- **Multi-input OR gates**(When either of it's N inputs has a value of 1, the output will be 1) are implemented because there are other control functions that will initiate similar operation.



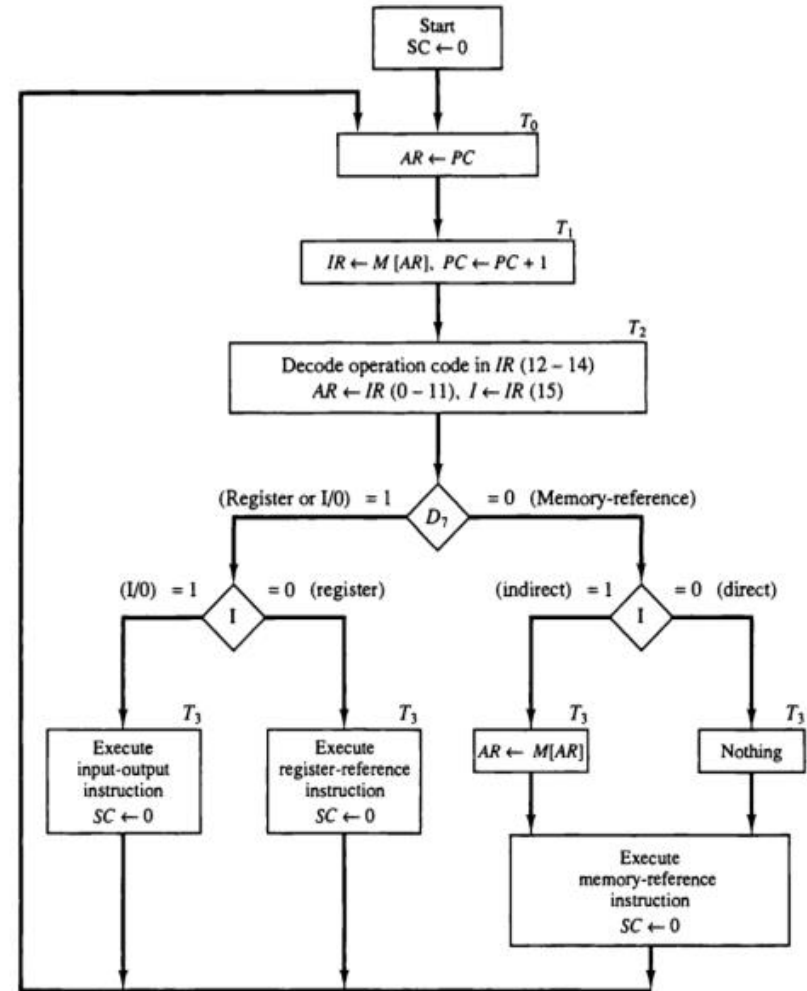
Flowchart of Instruction Cycle

- The timing signal that is active after the decoding is T_3 . **During time T_3** , the control unit **determines the type of instruction(instruction cycle)** that was just read from memory.
- The flowchart presents an initial configuration for the instruction cycle and shows how the control determines the instruction type after the decoding.



Flowchart of Instruction Cycle

- If $D_7 = 1$, if the operation code is equal to binary 111 and the instruction must be **register-reference** or **input-output type**.
- If $D_7 = 0$, the operation code must be one of the other seven values(000 to 110), specifying a **memory reference instruction**.
- Control then inspects the value of the first bit of the instruction, which now available in flip-flop I.
- If $D_7 = 0$ and $I = 1$, we have a memory-reference instruction with an indirect address. It is then necessary to read the effective address from memory. $AR \leftarrow M[AR]$



Flowchart of Instruction Cycle

- The three instruction types are subdivided into four separate paths. The selected operation is activated with the clock transition associated with timing signal T_3 . This can be symbolized as follows:

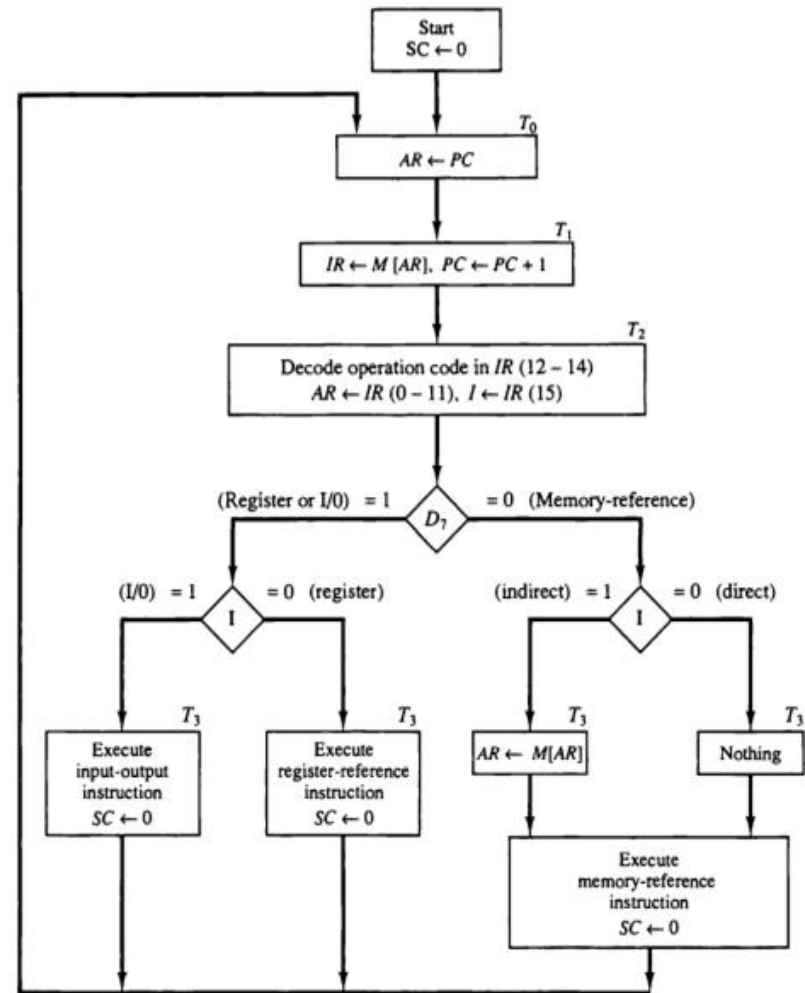
D_7IT_3 : $AR \leftarrow M[AR]$

$D_7I'T_3$: Nothing

$D_7I'T_3$: Execute a register-reference instruction

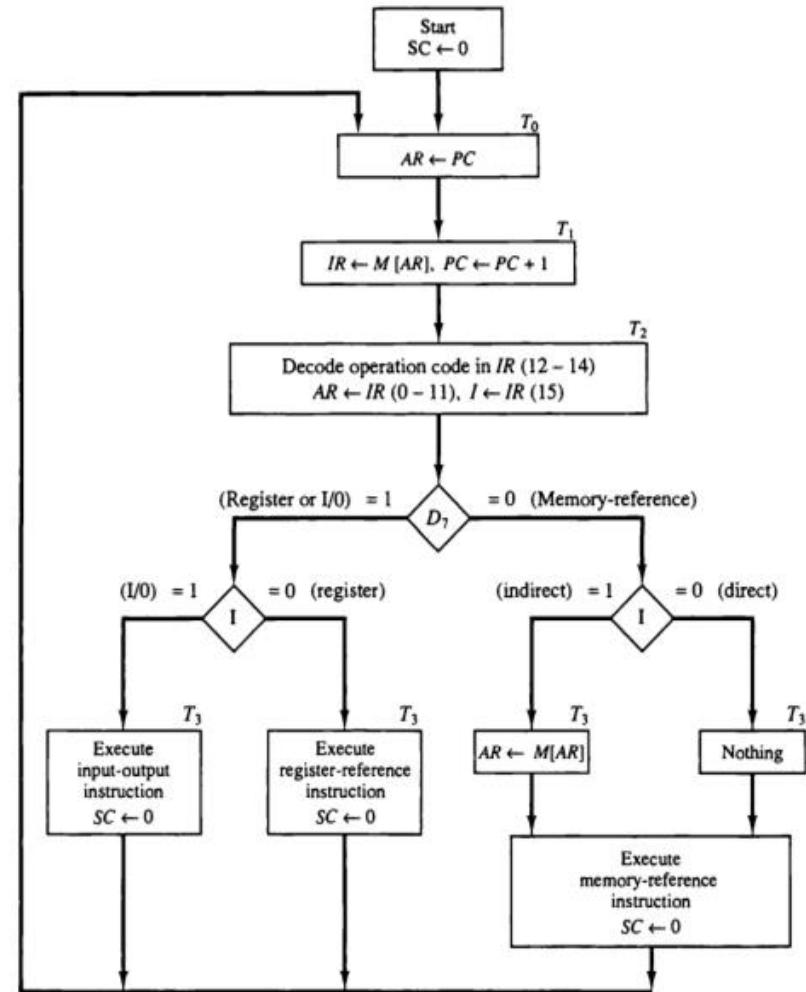
D_7IT_3 : Execute an input-output instruction

- When a memory-reference instruction with $I = 0$ is encountered, it is not necessary to do anything since the effective address is already in AR.



Flowchart of Instruction Cycle

- However, the sequence counter SC must be incremented when $D_7'IT_3 = 1$, so that the execution of the memory reference instruction can be continued with timing variable T_4 .
- A register-reference or input-output instruction can be executed with the clock associated with timing signal T_3 . After the instruction is executed, SC is cleared to 0 and control returns to the fetch phase with $T_0=1$.
- SC is either increment or cleared to 0 with every positive clock transition.



Register-Reference Instructions

Register Reference Instructions

- Register Reference Instructions are identified when, $D_7 = 1, I = 0$.
- These instructions use bits 0 to 11 of the instruction code to specify one of 12 instructions.
- These 12 bits are available in IR(0-11). They were also transferred to AR during time T_2 .
- Register Reference Instruction is specified in $B_0 \sim B_{11}$ of IR.
- These instructions are executed with the clock transition associated with timing signal variable T_3 .
- Each control function needs a boolean variable $D_7I'T_3$, which we designate by **symbol r**.
- The control function is distinguished by one of the bits in IR(0-11).
- By assigning the symbol B_i to bit i of IR, all control functions can be simply denoted by **rB_i** .
- The execution of a register-reference instruction is completed at time T_3 .
- The SC is cleared to 0 and control goes back to fetch the next instruction with timing signal T_0 .

Register Reference Instructions

- Example : CLA
- Hex code: 7800 Binary equivalent: 0111 1000 0000 0000.
- First bit is 0: I', Next three bits of opcode(111) generates D_7 and bit 11 in IR is 1 and is recognized from B_{11} .
- Control function for this instruction : $D_7I'T_3B_{11} = rB_{11}$.

$D_7I'T_3 = r$ (common to all register-reference instructions)

$IR(i) = B_i$ [bit in $IR(0-11)$ that specifies the operation]

CLA	rB_{11} :	$AC \leftarrow 0$	Clear AC
CLE	rB_{10} :	$E \leftarrow 0$	Clear E
CMA	rB_9 :	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8 :	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7 :	$AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	rB_6 :	$AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	rB_5 :	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4 :	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3 :	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2 :	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	rB_1 :	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0 :	$S \leftarrow 0$ (S is a start-stop flip-flop)	Halt computer

Memory-Reference Instructions

Memory Reference Instructions

- These instructions are useful to perform operations on operands. Operands will locate in main memory.
- The decoded output D_i for $i = 0, 1, 2, 3, 4, 5$, and 6 from the operation decoder that belongs to each instruction is included in the table.
- The effective address of the instruction is in the address register AR and was placed there during timing signal T_2 when $I = 0$, or during timing signal T_3 when $I = 1$.
- The execution of the memory-reference instructions starts with timing signal T_4 .

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], \quad E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

1. AND: AND to AC

- This is an instruction that performs the AND logic operation on pairs of bits in AC and the memory word specified by the effective address. The result of the operation is transferred to AC.

$D_0T_4: DR \leftarrow M[AR]$

$D_0T_5: AC \leftarrow AC \wedge DR, SC \leftarrow 0$

- The control function for this instruction uses the operation decoder D_0 since this output of the decoder is active when the instruction has an AND operation whose binary code value is 000.
- Two timing signals are needed to execute the instruction.
- The clock transition associated with timing signal T_4 transfers the operand from memory into DR .
- The clock transition associated with the next timing signal T_5 transfers to AC the result of the AND logic operation between the contents of DR and AC.
- The same clock transition clears SC to 0, transferring control to timing signal T_0 to start a new instruction cycle.

2. ADD: ADD to AC

- This instruction adds the content of the memory word specified by the effective address to the value of AC. The sum is transferred into AC and the output carry Cout is transferred to the E (extended accumulator) flip-flop.

$$D_1T_4: DR \leftarrow M[AR]$$

$$D_1T_5: AC \leftarrow AC + DR, \quad E \leftarrow C_{out}, \quad SC \leftarrow 0$$

- The control function for this instruction uses the operation decoder D_1 since this output of the decoder is active when the instruction has an ADD operation whose binary code value is 001.
- Two timing signals are needed to execute the instruction.
- The clock transition associated with timing signal T_4 transfers the operand from memory into DR .
- The clock transition associated with the next timing signal T_5 transfers to AC the result of the ADD operation between the contents of DR and AC.
- The same clock transition clears SC to 0, transferring control to timing signal T_0 to start a new instruction cycle.

3. **LDA: Load to AC**

- This instruction transfers the memory word specified by the effective address to AC.

$$D_2T_4: DR \leftarrow M[AR]$$

$$D_2T_5: AC \leftarrow DR, SC \leftarrow 0$$

- There is no direct path from the bus into AC .
- The adder and logic circuit receive information from DR which can be transferred into AC
- It is necessary to read the memory word into DR first and then transfer the content of DR into AC .

4. **STA: Store AC**

- This instruction stores the content of AC into the memory word specified by the effective address.
- Since the output of AC is applied to the bus and the data input of memory is connected to the bus.

$$D_3T_4: M[AR] \leftarrow AC, SC \leftarrow 0$$

5. **BUN: Branch Unconditionally**

- This instruction transfers the program to instruction specified by the effective address.
- PC holds the address of the instruction to be read from memory in the instruction cycle.
- The BUN instruction allows the programmer to specify an instruction out of sequence and the program branches (or jumps) unconditionally.

$$D_4T_4: PC \leftarrow AR, SC \leftarrow 0$$

- The effective address from AR is transferred through the common bus to PC. Resetting SC to 0 transfers control to T0.
- The next instruction is then fetched and executed from the memory address given by the new value in PC.

Example:

BUN 0011H

6. BSA: Branch and Save Return Address

- This instruction is useful for branching to a portion of the program called a subroutine or procedure.
- When executed, the BSA instruction stores the address of the next instruction in sequence (which is available in PC) into a memory location specified by the effective address.
- The effective address plus one is then transferred to PC to serve as the address of the first instruction in the subroutine.

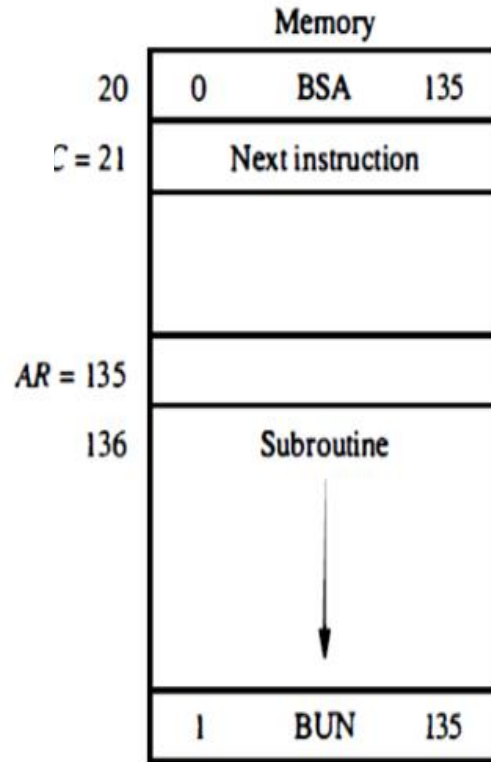
$$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$$

- The BSA instruction performs the function usually referred to as a subroutine call.
- The indirect BUN instruction at the end of the subroutine performs the function referred to as a subroutine return.

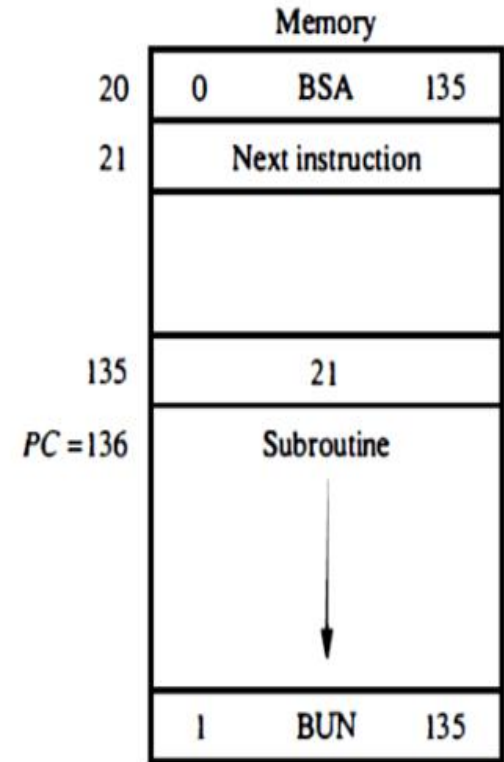
BSA-Example

$D_5T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$
 $D_5T_5: PC \leftarrow AR, SC \leftarrow 0$

- Timing signal T4 initiates a memory write operation, places the content of PC onto the bus, and enables the INR input of AR.
- The memory write operation is completed and AR is incremented by the time the next clock transition occurs.
- The bus is used at T5 to transfer content of AR to PC.
- $M[135] \leftarrow 21, AR \leftarrow 135 + 1 = 136$**
- $PC \leftarrow 136, SC \leftarrow 0$**



(a) Memory, PC, and AR at time T_4



(b) Memory and PC after execution

7. ISZ: Increment and Skip if Zero

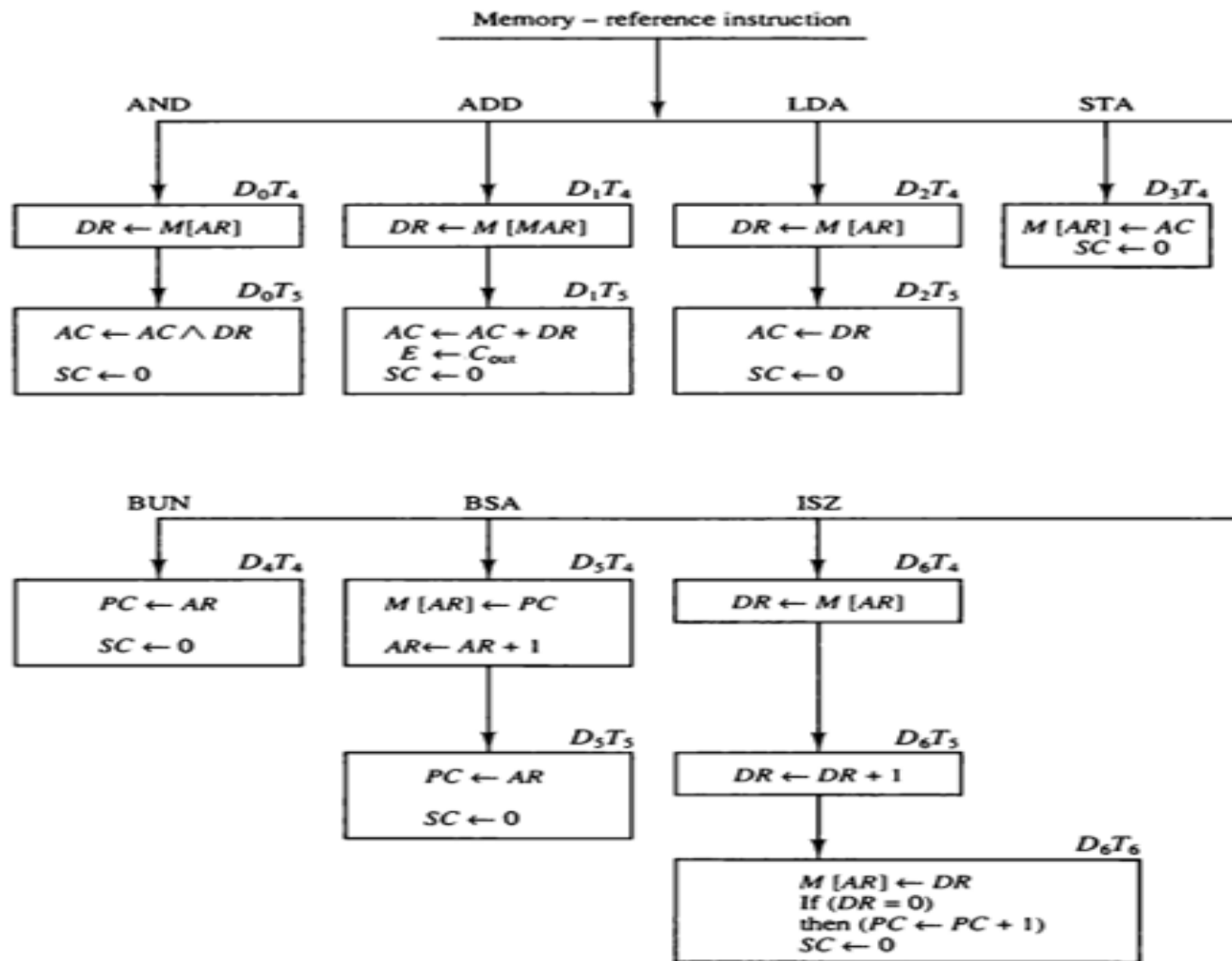
- This instruction increments the word(data) specified by the effective address, and if the incremented value is equal to 0, PC is incremented by 1.
- The programmer usually stores a (-ve) number in 2's complement in the memory word. As this negative number is repeatedly incremented by one, it eventually reaches the value of zero. At that time PC is incremented by one in order to skip the next instruction in the program.
- Since it is not possible to increment a word inside the memory, it is necessary to read the word into DR, increment DR, and store the word back into memory.

$D_6T_4: DR \leftarrow M[AR]$

$D_6T_5: DR \leftarrow DR + 1$

$D_6T_6: M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), \quad SC \leftarrow 0$

Flowchart of Memory Reference Instructions



Input-output and interrupt

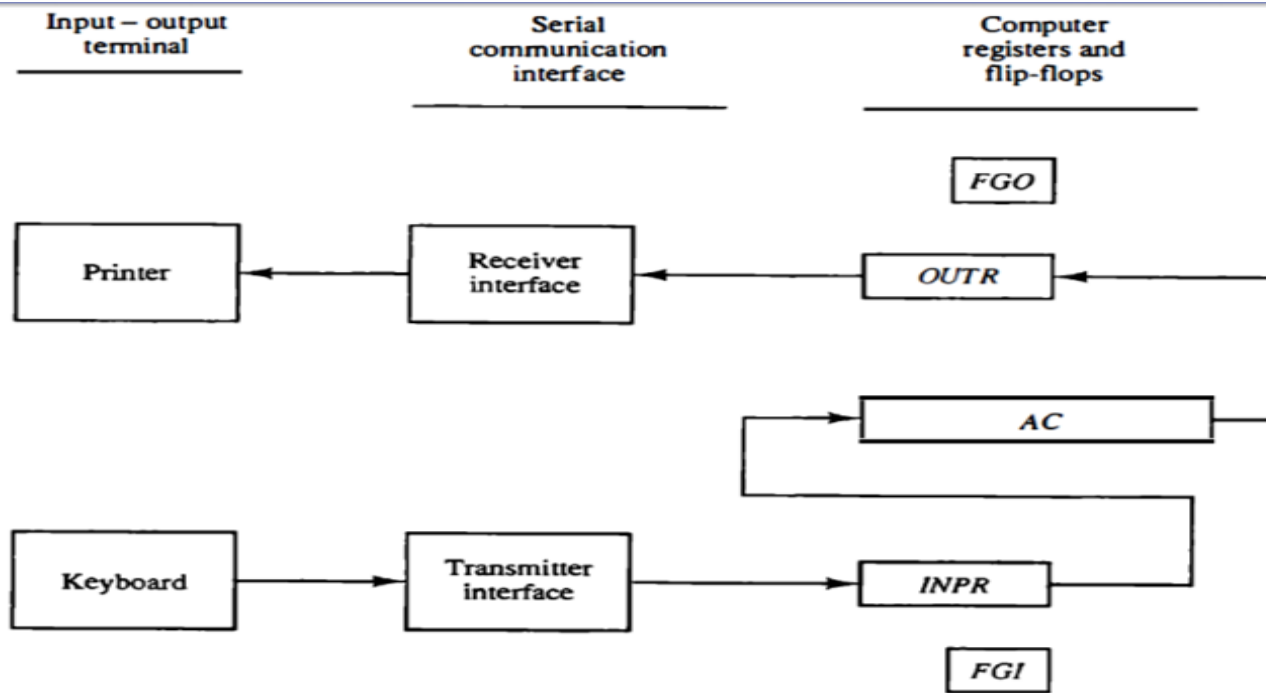
Input – Output and Interrupt

- A computer can serve no useful purpose unless it communicates with the external environment.
- Instructions and data stored in memory must come from some input device.
- Computational results must be transmitted to the user through some output device.
- To demonstrate the most basic requirements for input and output communication, we will use a terminal unit with a keyboard and printer.

Input – Output Configuration

- The terminal sends and receives serial information and each quantity of information has eight bits of an alphanumeric code.
- The serial information from the keyboard is shifted into the input register INPR.
- The serial information for the printer is stored in the output register OUTR.
- These two registers communicate with a communication interface serially and with the AC in parallel.
- The transmitter interface receives serial information from the keyboard and transmits it to INPR.
- The receiver interface receives information from OUTR and sends it to the printer serially

Input – Output Configuration



Initially FGI=0

Keyboard to INPR
INPR to AC

FGI=1
FGI=0

Initially FGO=1

AC to OUTR
OUTR to Printer

FGO=0
FGO=1

❖ **Input Register:**

- The input register INPR consists of eight bits and holds an alphanumeric input information.
- The 1-bit input flag FGI is a control flip-flop. It is set to 1 when new information is available in the input device and is cleared to 0 when the information is accepted by the computer.
- The flag is needed to synchronize the timing rate difference between the input device and the computer.
- **Process of information transfer:**
- Initially, the input flag FGI is cleared to 0. When a key is struck in the keyboard, an 8-bit alphanumeric code is shifted into INPR and the input flag FGI is set to 1.
- As long as the flag is set, the information in INPR cannot be changed by striking another key, the computer checks the flag bit if it is 1, the information from INPR is transferred in parallel into AC and FGI is cleared to 0.
- Once the flag is cleared, new information can be shifted into INPR by striking another key.

❖ **Output Register:**

- The output register OUTR works similarly but the direction of information flow is reversed.
- Initially, the output flag FGO is set to 1. The computer checks the flag bit if it is 1, the information from AC is transferred in parallel to OUTR and FGO is cleared to 0.
- The output device accepts the coded information, prints the corresponding character, and when the operation is completed, it sets FGO to 1.
- The computer does not load a new character into OUTR when FGO is 0 because this condition indicates that the output device is in the process of printing the character.

Input-Output Instructions

- Input and output instructions are needed for transferring information to and from AC register, for checking the flag bits, and for controlling the interrupt facility.
- Input-output instructions have an operation code 1111 and are recognized by the control when $D_7=1$ and $I=1$. The remaining bits of the instruction specify the particular operation.

$D_7IT_3 = p$ (common to all input-output instructions)

$IR(i) = B_i$ [bit in $IR(6-11)$ that specifies the instruction]

INP	pB_{11} :	$AC(0-7) \leftarrow INPR, \quad FGI \leftarrow 0$	Input character
OUT	pB_{10} :	$OUTR \leftarrow AC(0-7), \quad FGO \leftarrow 0$	Output character
SKI	pB_9 :	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$	Skip on input flag
SKO	pB_8 :	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$	Skip on output flag
ION	pB_7 :	$IEN \leftarrow 1$	Interrupt enable on
IOF	pB_6 :	$IEN \leftarrow 0$	Interrupt enable off

Input-Output Instructions

- The INP instruction transfers the input information from INPR into the eight low-order bits of AC and also clears the input flag to 0.
- The OUT instruction transfers the eight least significant bits of AC into the output register OUTR and clears the output flag to 0.
- The next two instructions in Table check the status of the flags and cause a skip of the next instruction if the flag is 1.
- The instruction that is skipped will normally be a branch instruction to return and check the flag again.
- The branch instruction is not skipped if the flag is 0. If the flag is 1, the branch instruction is skipped and an input or output instruction is executed.
- The last two instructions set and clear an interrupt enable flipflop IEN. The purpose of IEN is explained in conjunction with the interrupt operation.

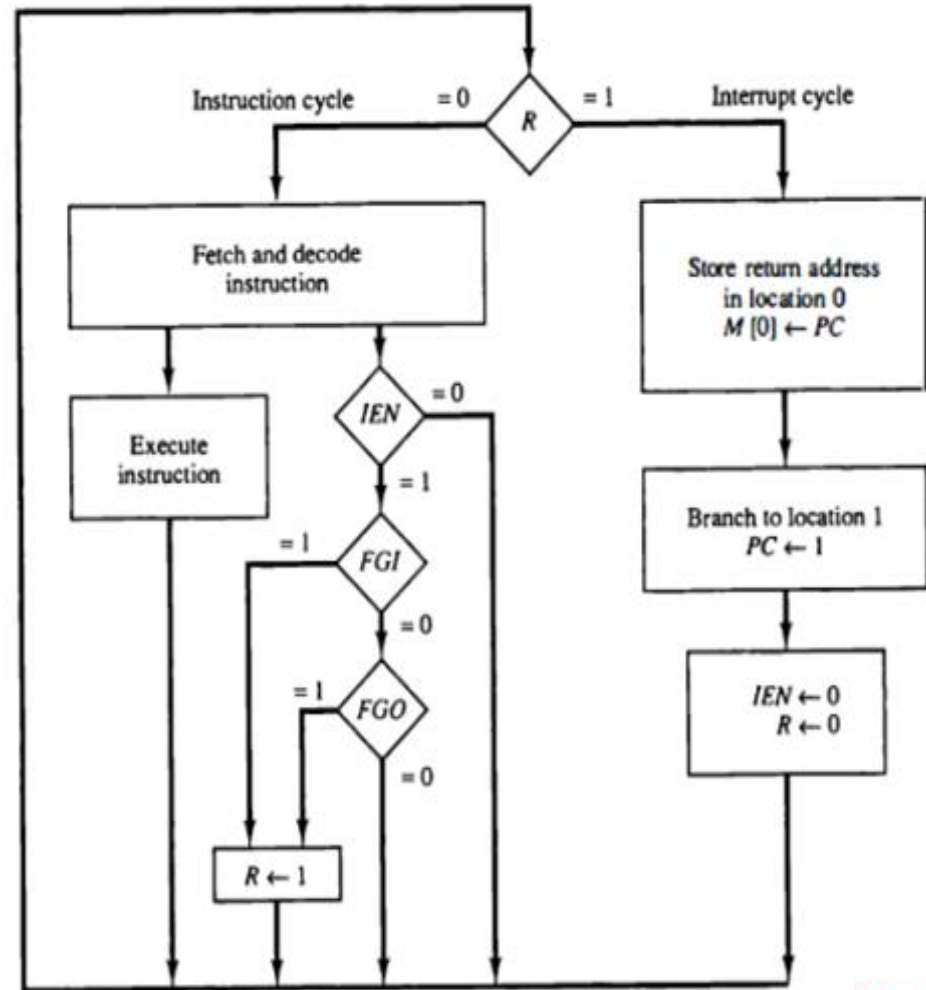
Interrupt Cycle

Interrupt Cycle

- **Interrupt:** It is a special signal which needs to be executed immediately.
- Whenever CPU receives interrupt signal then CPU stops execution of the currently running program, CPU control will be shifted to the interrupt related program. So, once CPU completed the execution of the interrupt related program, CPU control will be shifted to the original program.
- The interrupt enable flip-flop IEN can be set and cleared with two instructions.
- When IEN is cleared to 0 (with the IOF instruction), the flags cannot interrupt the computer.
- When IEN is set to 1 (with the ION instruction), the computer can be interrupted.
- These two instructions provide the programmer with the capability of making a decision as to whether or not to use the interrupt facility.

Interrupt Cycle:

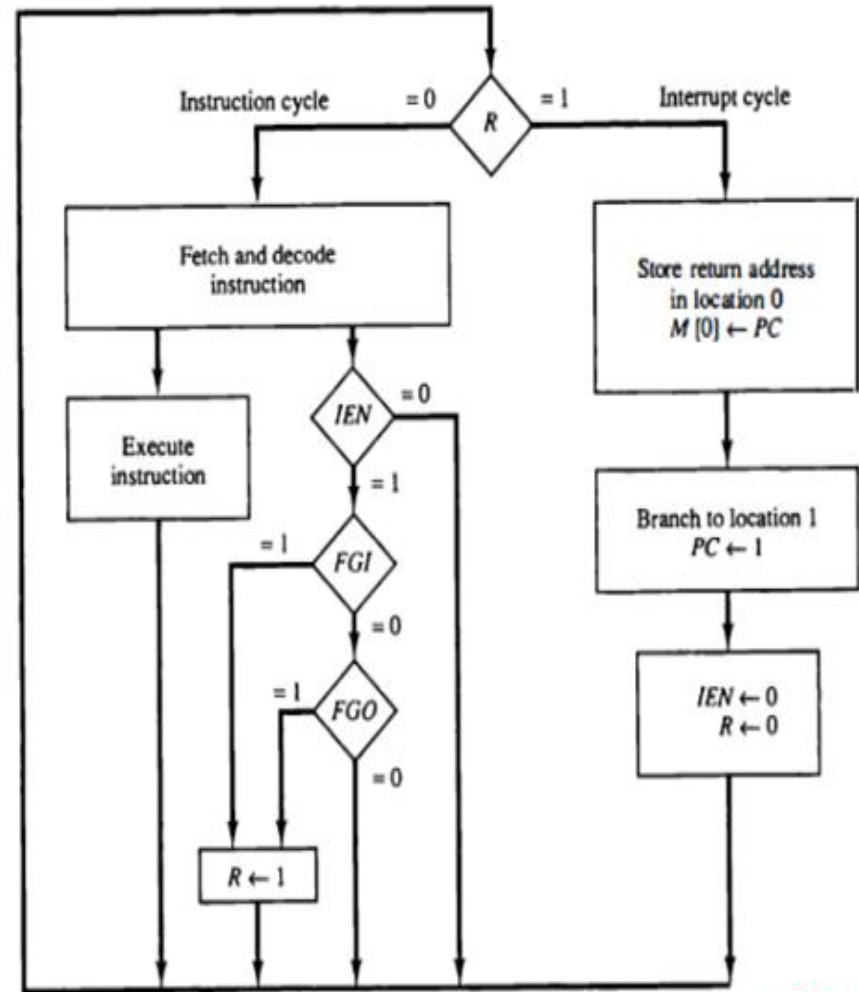
- An interrupt flip-flop R is included in the computer.
- When $R = 0$, the computer goes through an instruction cycle. During the execute phase of the instruction cycle IEN is checked by the control.
- If IEN is 0, it indicates that the programmer does not want to use the interrupt, so control continues with the next instruction cycle.
- If IEN is 1, control checks the flag bits. If both flags are 0, it indicates that neither the input nor the output registers are ready for transfer of information.
- In this case, control continues with the next instruction cycle. If either flag is set to 1 while IEN=1, flip-flop R is set to 1.
- At the end of the execute phase, control checks the value of R .



Cont...

Interrupt Cycle:

- If R is equal to 1, the computer goes to an interrupt cycle instead of an instruction cycle.
- The **interrupt cycle is a hardware implementation of a branch and save return address operation.**
- The **return address available in PC is stored in a specific location** where it can be found later when the program returns to the instruction at which it was interrupted.
- This location may be a **processor register, a memory stack or a specific memory location.**
- **Select the memory location at address 0 as the place for storing the address.** Control then inserts address 1 into PC and clears IEN and R so that no more interruption can occur until the interrupt request from the flag has been serviced.



Cont...

- **Register Transfer Statements for Interrupt Cycle**
- Condition for setting flip-flop R to 1:

$$T_0' T_1' T_2' (IEN) (FGI + FGO): R \leftarrow 1$$


 OR

- Interrupt Cycle Statement:

$RT_0: AR \leftarrow 0, TR \leftarrow PC$

$RT_1: M[AR] \leftarrow TR, PC \leftarrow 0$

$RT_2: PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$

Example:

- Suppose that an interrupt occurs and $R = 1$, while the control is executing the instruction at address 255. At this time, the return address 256 is in PC.
- The programmer has previously placed an input-output service program in memory starting from address 1120 and a BUN 1120 instruction at address 1.
- The content of PC (256) is stored in memory location 0, PC is set to 1, and R is cleared to 0.
- At the beginning of the next instruction cycle, the instruction that is read from memory is in address 1 since this is the content of PC. The branch instruction at address 1 causes the program to transfer to the input-output service program at address 1120.
- This program checks the flags, determines which flag is set, and then transfers the required input or output information. Once this is done, the instruction ION is executed to set IEN to 1 (to enable further interrupts), and the program returns to the location where it was interrupted.
- The instruction that returns the computer to the original place in the main program is a branch indirect instruction with an address part of 0. This instruction is placed at the end of the I/O service program.
- The execution of the indirect BUN instruction results in placing into PC the return address from location 0.

RT₀: AR ← -0, TR ← -256

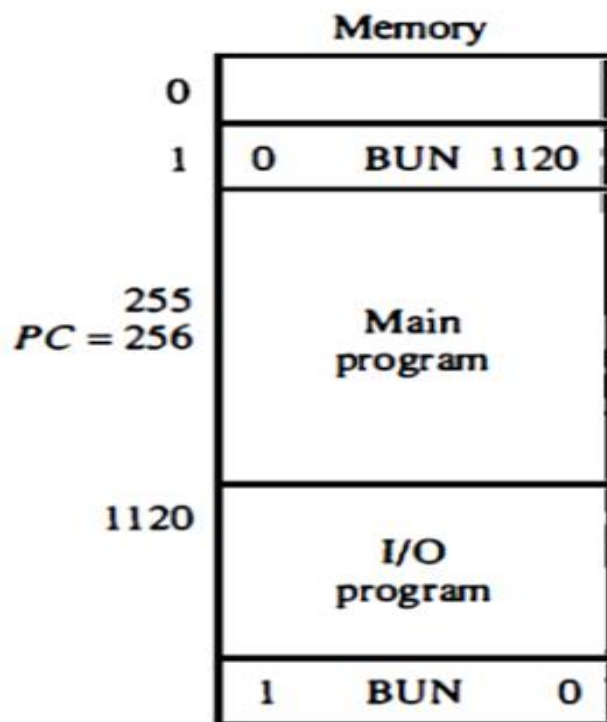
RT₁: M[0] ← -256, PC ← -0

RT₂: PC ← -PC + 1, IEN ← -0, R ← -0, SC ← -0

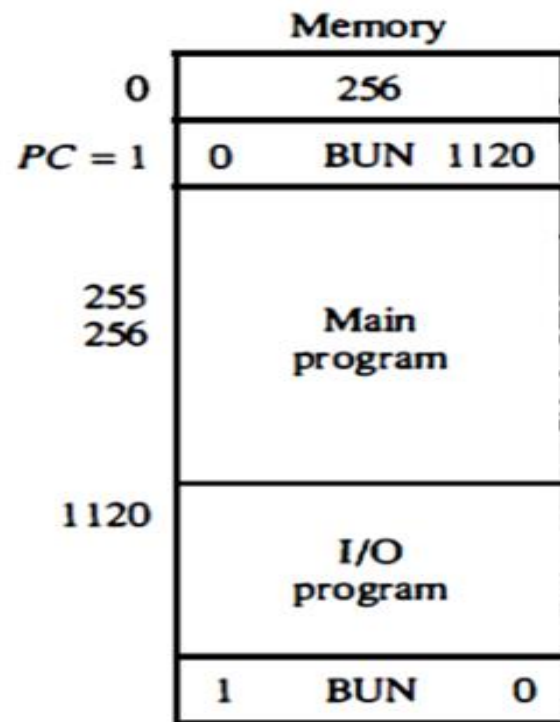
RT₀: AR ← 0, TR ← PC

RT₁: M[AR] ← TR, PC ← 0

RT₂: PC ← PC + 1, IEN ← 0, R ← 0, SC ← 0



(a) Before interrupt

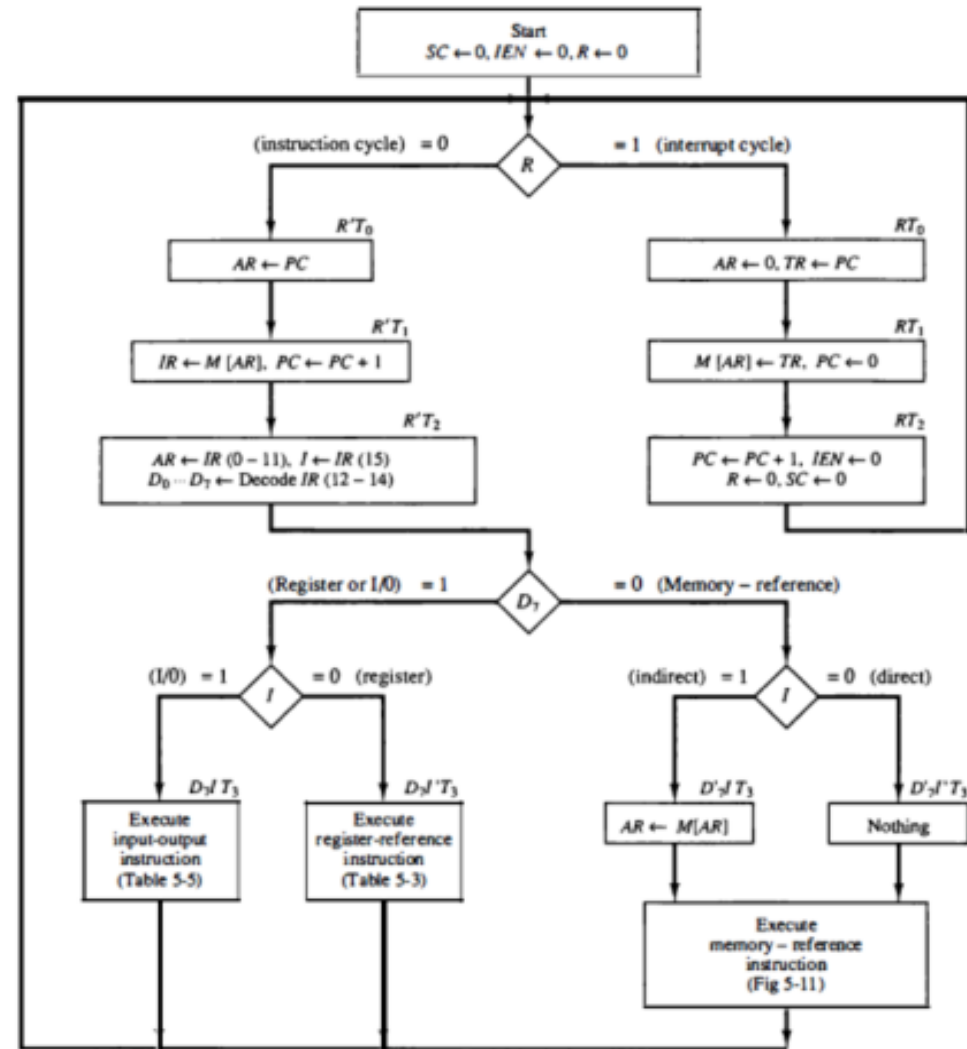


(b) After interrupt cycle

Design of Basic computer

Design of Basic Computer

- The basic computer consists of the following hardware components:
- A memory unit with 4096 words of 16 bits each
- Nine registers: AR, PC, DR, AC, IR, TR, OTR, INPR, and SC
- Seven flip-flops: I, S, E, R, IEN, FGI, and FGO
- Two decoders: a 3 x 8 operation decoder and a 4 x 16 timing decoder
- A 16-bit common bus
- Control logic gates
- Adder and logic circuit connected to the input of AC



Design of Accumulator Logic

Design of Accumulator Logic

Fetch	$R'T_0$:	$AR \leftarrow PC$
	$R'T_1$:	$IR \leftarrow M[AR], \quad PC \leftarrow PC + 1$
Decode	$R'T_2$:	$D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14),$ $AR \leftarrow IR(0-11), \quad I \leftarrow IR(15)$
Indirect	D_7IT_3 :	$AR \leftarrow M[AR]$
Interrupt:		
	$T_0T_1T_2(IEN)(FGI + FGO)$:	$R \leftarrow 1$
	RT_0 :	$AR \leftarrow 0, \quad TR \leftarrow PC$
	RT_1 :	$M[AR] \leftarrow TR, \quad PC \leftarrow 0$
	RT_2 :	$PC \leftarrow PC + 1, \quad IEN \leftarrow 0, \quad R \leftarrow 0, \quad SC \leftarrow 0$
Memory-reference:		
AND	D_0T_4 :	$DR \leftarrow M[AR]$
	$\longrightarrow D_0T_5$:	$AC \leftarrow AC \wedge DR, \quad SC \leftarrow 0$
ADD	D_1T_4 :	$DR \leftarrow M[AR]$
	$\longrightarrow D_1T_5$:	$AC \leftarrow AC + DR, \quad E \leftarrow C_{out}, \quad SC \leftarrow 0$
LDA	D_2T_4 :	$DR \leftarrow M[AR]$
	$\longrightarrow D_2T_5$:	$AC \leftarrow DR, \quad SC \leftarrow 0$
STA	D_3T_4 :	$M[AR] \leftarrow AC, \quad SC \leftarrow 0$
BUN	D_4T_4 :	$PC \leftarrow AR, \quad SC \leftarrow 0$
BSA	D_5T_4 :	$M[AR] \leftarrow PC, \quad AR \leftarrow AR + 1$
	D_5T_5 :	$PC \leftarrow AR, \quad SC \leftarrow 0$
ISZ	D_6T_4 :	$DR \leftarrow M[AR]$
	D_6T_5 :	$DR \leftarrow DR + 1$
	D_6T_6 :	$M[AR] \leftarrow DR, \quad \text{if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), \quad SC \leftarrow 0$

Design of Accumulator Logic

Register-reference:

$D_7I'T_3 = r$ (common to all register-reference instructions)

$IR(i) = B_i$ ($i = 0, 1, 2, \dots, 11$)

	$r:$	$SC \leftarrow 0$
CLA	$\longrightarrow rB_{11}:$	$AC \leftarrow 0$
CLE	$rB_{10}:$	$E \leftarrow 0$
CMA	$\longrightarrow rB_9:$	$AC \leftarrow \overline{AC}$
CME	$rB_8:$	$E \leftarrow \overline{E}$
CIR	$\longrightarrow rB_7:$	$AC \leftarrow \text{shr } AC, \quad AC(15) \leftarrow E, \quad E \leftarrow AC(0)$
CIL	$\longrightarrow rB_6:$	$AC \leftarrow \text{shl } AC, \quad AC(0) \leftarrow E, \quad E \leftarrow AC(15)$
INC	$\longrightarrow rB_5:$	$AC \leftarrow AC + 1$
SPA	$rB_4:$	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$
SNA	$rB_3:$	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$
SZA	$rB_2:$	If $(AC = 0)$ then $PC \leftarrow PC + 1$
SZE	$rB_1:$	If $(E = 0)$ then $(PC \leftarrow PC + 1)$
HLT	$rB_0:$	$S \leftarrow 0$

Input-output:

$D_7IT_3 = p$ (common to all input-output instructions)

$IR(i) = B_i$ ($i = 6, 7, 8, 9, 10, 11$)

	$p:$	$SC \leftarrow 0$
INP	$\longrightarrow pB_{11}:$	$AC(0-7) \leftarrow INPR, \quad FGI \leftarrow 0$
OUT	$pB_{10}:$	$OUTR \leftarrow AC(0-7), \quad FGO \leftarrow 0$
SKI	$pB_9:$	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$
SKO	$pB_8:$	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$
ION	$pB_7:$	$IEN \leftarrow 1$
IOF	$pB_6:$	$IEN \leftarrow 0$

Design of Accumulator Logic

- In order to design the logic associated with AC, it is necessary to extract all the statements that change the content of AC:

$D_0T_5:$	$AC \leftarrow AC \wedge DR$	AND with DR
$D_1T_5:$	$AC \leftarrow AC + DR$	Add with DR
$D_2T_5:$	$AC \leftarrow DR$	Transfer from DR
$pB_{11}:$	$AC(0-7) \leftarrow INPR$	Transfer from $INPR$
$rB_9:$	$AC \leftarrow \overline{AC}$	Complement
$rB_7:$	$AC \leftarrow \text{shr } AC, \quad AC(15) \leftarrow E$	Shift right
$rB_6:$	$AC \leftarrow \text{shl } AC, \quad AC(0) \leftarrow E$	Shift left
$rB_{11}:$	$AC \leftarrow 0$	Clear
$rB_5:$	$AC \leftarrow AC + 1$	Increment

Design of Accumulator Logic

Figure 5-19 Circuits associated with AC.

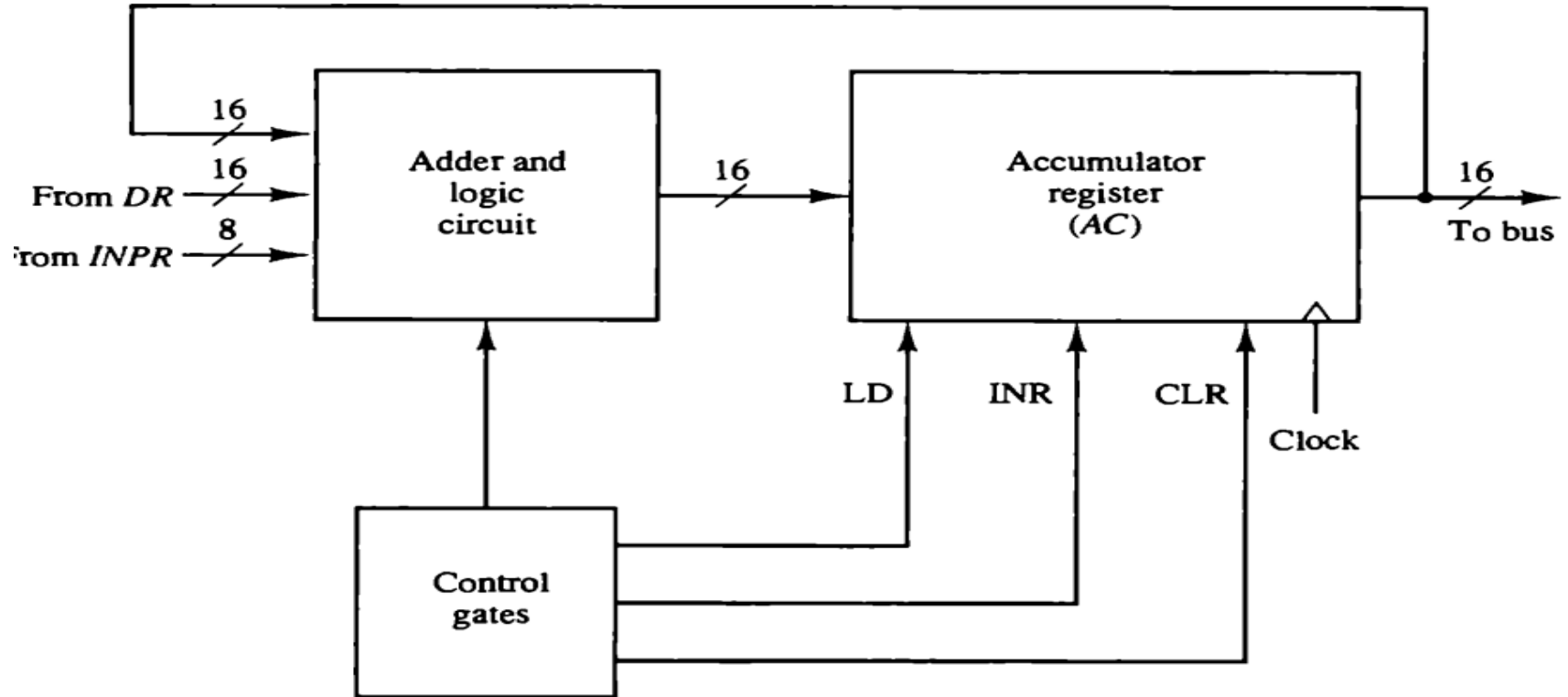
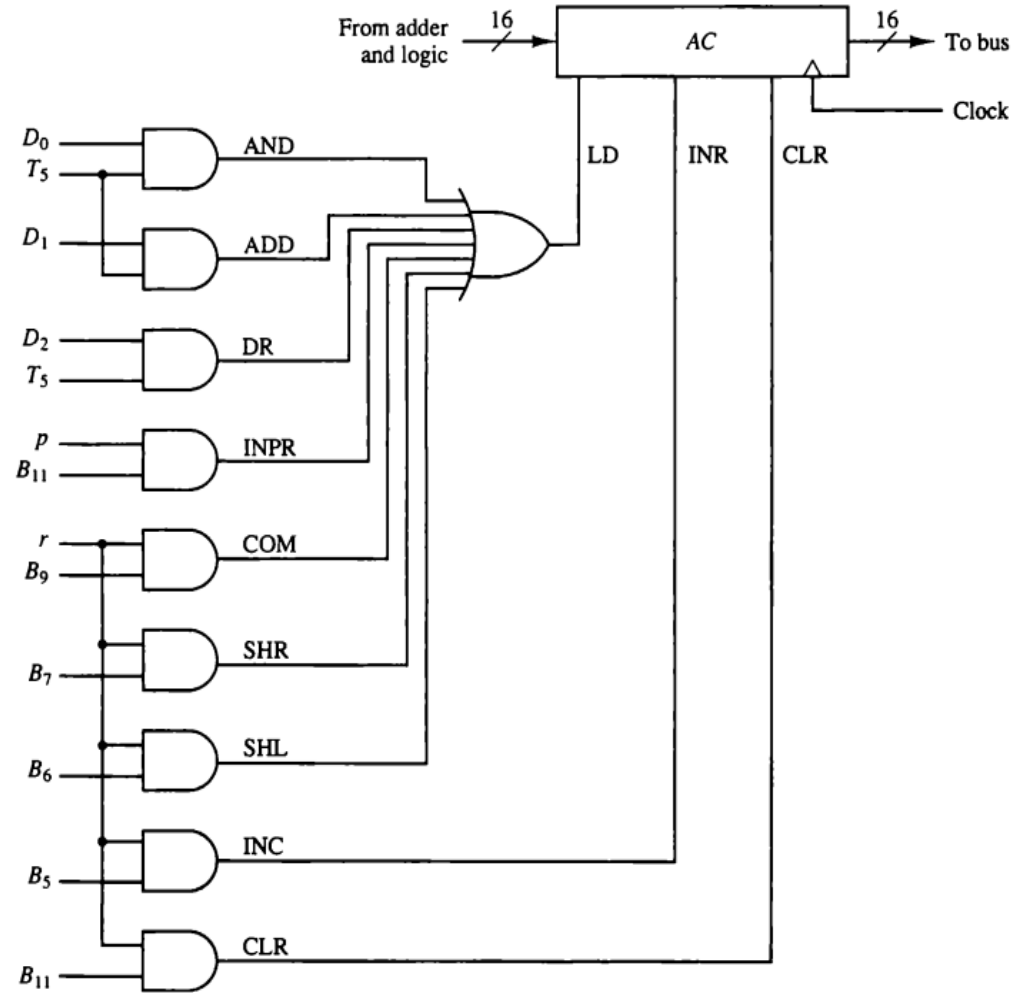


Figure 5-20 Gate structure for controlling the LD, INR, and CLR of AC.



Design of Accumulator Logic

GTU Questions

Q.1 Enlist register reference instructions and explain any three of them in detail.

OR

Explain register reference instruction format.

Q.2 Draw the flowchart for instruction cycle and explain.

OR

Explain Instruction cycle with flowchart.

OR

Describe how control unit determine instruction type after the decoding using flowchart for instruction cycle.

Q.3 One hypothetical basic computer has the following specifications: Addressing Mods = 16 Total Instruction Types = 4 (IT1, IT2, IT3, IT4) Each of the instruction type has 16 different instructions. Total General-Purpose Register = 8 Size of Memory = 8192×8 bits Maximum number of clock cycles required to execute one instruction = 32 Each instruction of the basic computer has one memory operand and one register operand in addition to other required fields.

a. Draw the instruction word format and indicate the number of bits in each part.

b. Draw the block diagram of control unit.

Q.4 Write down RTL statements for the fetch and decode operation of basic computer.

Q.5 List and explain Memory reference instructions in detail.

OR

Explain memory reference instructions of 'Basic Computer'.

GTU Questions

Q.6 List out Register for basic computer.

OR

Write the name of basic computer registers with their functionalities.

OR

List out names of eight main registers of basic computer with their symbolic name and purpose.

OR

List all registers used in 'Basic Computer' including their symbolic name, size and functionality.

Q.7 Draw and explain control unit diagram for basic computer.

OR

What is the role of sequence counter(SC) in control unit? Interpret its concept with the help of its three inputs using diagram.

Q.8 State various phases of instruction cycle.

Q.9 Explain any four input output reference instruction.

Q.10 Write sequence of microoperations to execute the following instructions: - AND – STA

Q.11 A computer uses a memory unit with 256K words of 32 bits each. A binary instruction code is stored in one word of memory. The instruction has four parts: an indirect bit, an operation code, a register code part to specify one of 64 registers, and an address part.

- 1. How many bits are there in operation code, the register code part and the address part?**
- 2. Draw the instruction word format and indicate the number of bits in each part.**
- 3. How many bits are there in the data and address inputs of the memory?**

GTU Questions

Q.12 Differentiate MRI and non-MRI.

Q.13 Explain CLA, ISZ and CMA instruction.

Q.14 Draw and explain in brief flowchart for interrupt cycle.

OR

What is interrupt? Describe interrupt cycle with neat diagram.

Q.15 Describe BUN and BSA memory reference instructions in detail.

OR

Use BSA and BUN instruction with example and diagram.

Q.16 What is the need of common bus? Draw common bus cycle.

Q.17 Which are the different phases of Instruction Cycle? Describe Register transfer for fetch phase with its diagram.

Q.19 Interpret the following instructions: INP, ISZ and LDA

Q.20 Calculate and show the number of clock cycles required to execute BSA instruction.

Q.21 Show the working of LDA instruction using RTL.

GTU Questions

Q.3 One hypothetical basic computer has the following specifications: Addressing Mods = 16 Total Instruction Types = 4 (IT1, IT2, IT3, IT4) Each of the instruction type has 16 different instructions. Total General-Purpose Register = 8 Size of Memory = 8192 X 8 bits Maximum number of clock cycles required to execute one instruction = 32 Each instruction of the basic computer has one memory operand and one register operand in addition to other required fields.

- a. Draw the instruction word format and indicate the number of bits in each part.
- b. Draw the block diagram of control unit.

Opcode	Addressing Mode	Memory Address	Register Operand	Other
Instruction (4*16) (6 bits)	4 bits	8192 memory locations (13 bits)	8 registers (3 bits)	6 bits

- The total number of bits in the instruction word format is 32 bits, which aligns with the requirement of 32 clock cycles to execute one instruction.
- Hint for the Control Unit: 1) Decoder Size: 6×64 2) Decoder Size: 5×32

GTU Questions

Q.4 Write down RTL statements for the fetch and decode operation of basic computer.

- Microoperations for the fetch and decode phases:

Fetching T_0 : $AR \leftarrow PC$

Fetching T_1 : $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$

Decoding T_2 : $AR \leftarrow IR(0-11)$, $D0, \dots, D7 \leftarrow \text{Decode } IR(12-14)$, $I \leftarrow IR(15)$

GTU Questions

Q.11 A computer uses a memory unit with 256K words of 32 bits each. A binary instruction code is stored in one word of memory. The instruction has four parts: an indirect bit, an operation code, a register code part to specify one of 64 registers, and an address part.

- 1. How many bits are there in operation code, the register code part and the address part?
- 2. Draw the instruction word format and indicate the number of bits in each part.
- 3. How many bits are there in the data and address inputs of the memory?

Indirect bit(1 bit)	Operation Code (17 bits)	Register code (6 bits)	Address (18 bits)
I (1 bit)	7 bits	64 registers (6 bits)	256K words= 256×1024 words $2^8 \times 2^{10}$ (18 bits)

Opcode bits = $32 - (8 + 6 + 1) = 7$

Data inputs: 32 bits

Address input: 18 bits

GTU Questions

Q.12 Differentiate MRI and non-MRI.

Feature	MRI (Memory Reference Instruction)	Non-MRI (Non-Memory Reference Instruction)
Definition	Instructions that access memory during execution.	Instructions that do not directly access memory.
Usage	Used for data transfer, load/store, and arithmetic operations involving memory.	Used for control, input/output (I/O), and register-based operations.
Examples	LDA (Load Accumulator), STA (Store Accumulator), ADD (Addition with memory operand).	HLT (Halt), CLA (Clear Accumulator), INP (Input), OUT (Output).
Memory Interaction	Directly interacts with memory to fetch or store data.	Operates only on registers or performs control tasks without memory access.
Execution Time	Takes more time as it involves memory access.	Generally faster since it avoids memory fetch/store operations.

GTU Questions

Q.20 Calculate and show the number of clock cycles required to execute BSA instruction.

1. Fetch Cycle (Instruction Fetch):

- The instruction is fetched from memory.
- The Program Counter (PC) is incremented.
- Requires **2 clock cycles**.
- Fetching T_0 : $AR \leftarrow PC$
- Fetching T_1 : $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$

2. Decode Cycle (Instruction Decode):

- Instruction is decoded.
- Requires **1 clock cycle**.
- Decoding T_2 : $AR \leftarrow IR(0-11)$, $D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14)$, $I \leftarrow IR(15)$

3. Indirect Addressing (if applicable):

- If indirect addressing is used, an additional memory access is needed.
- Requires **1 clock cycle (if indirect)**.
- T_3 : $AR \leftarrow M[AR]$

GTU Questions

Q.20 Calculate and show the number of clock cycles required to execute BSA instruction.

4. Store Return Address:

- The current PC is stored at the address specified in the instruction.
- This involves a memory write operation.
- Requires **1 clock cycle**.
- $T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$

5. Load New PC (Branch):

- The PC is updated to the subroutine's address.
- Requires **1 clock cycle**.
- $T_5 PC \leftarrow AR.$

Total Clock Cycles	
Description	Clock Cycles
Fetch instruction	2
Decode instruction	1
Indirect Addressing	1(optional)
Store Return Address	1
Load New PC	1
Without indirect addressing	5
With indirect addressing	6

Thanking You